

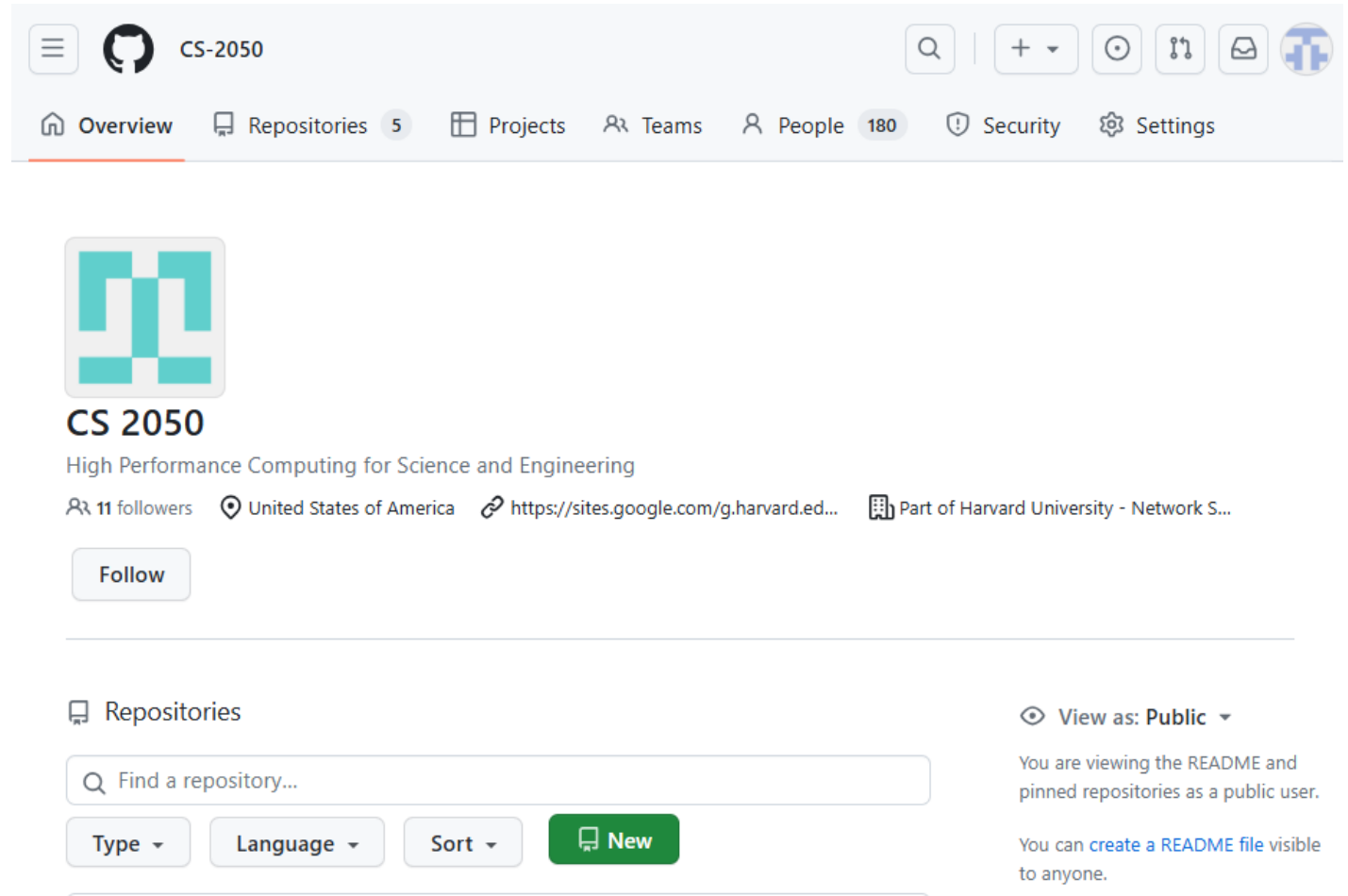
HIGH PERFORMANCE COMPUTING FOR SCIENCE AND ENGINEERING

Harvard University
CS 2050

Lecture 3
February 3, 2026

CS-2050 GITHUB ORGANIZATION

- If you haven't used `code.harvard.edu` previously, please **log on now**
- Your account won't exist until you log on for the first time
- We can't add you to CS-2050 until your account exists



 **questions-and-answers** Public

 Edit Pins ▾

 Watch 0 ▾

 Fork 0 ▾

 Star 0 ▾

 main ▾





Q Go to file t

+

<> Code ▾

 **wcw398** update readme

4be70a2 · 17 hours ago

 4 Commits

 README.md

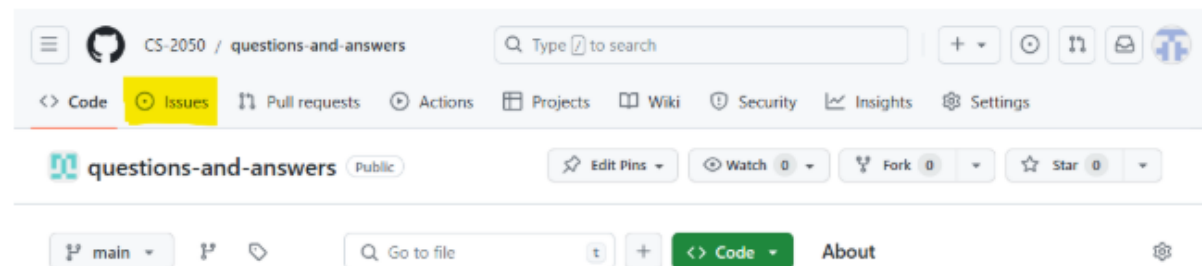
update readme

17 hours ago

 README

Questions and Answers

Please create issues in this repository to ask questions about the course.



About

Ask your CS 2050 questions here.

 sites.google.com/g.harvard.edu/cs-2050

 Readme

 Activity

 Custom properties

 0 stars

 0 watching

 0 forks

Releases

No releases published

[Create a new release](#)

Deadlines

February

Tue Feb 03 - [Quiz 1](#) due by 11:59pm

Tue Feb 10 - Quiz 2 due by 11:59pm

Thu Feb 12 - Homework 1 due by 11:59pm

Tue Feb 17 - Quiz 3 due by 11:59pm

Tue Feb 24 - Quiz 4 due by 11:59pm

Thu Feb 26 - Homework 2 due by 11:59pm

QUIZ 1

COMPSCI 2050 > Quizzes

2025-2026 Spring

Search for Quiz

▼ Assignment Quizzes

Quiz 1
Available until Feb 3 at 11:59pm Due Feb 3 at 11:59pm 14 pts 14 Questions

Copyright © 2013-2026 The President and Fellows of Harvard College | [Acceptable Use Policy](#) | [Accessibility](#) | [Digital Accessibility](#) | [Privacy Policy](#)

- Don't click on the quiz until you are ready to take it

HOMework 1

- Stay tuned for Homework 1
- Deadline: Thurs Feb 12, end of day

OFFICE HOURS

- Temporary location:
SEC, Room 1.312
- Or Zoom where
indicated

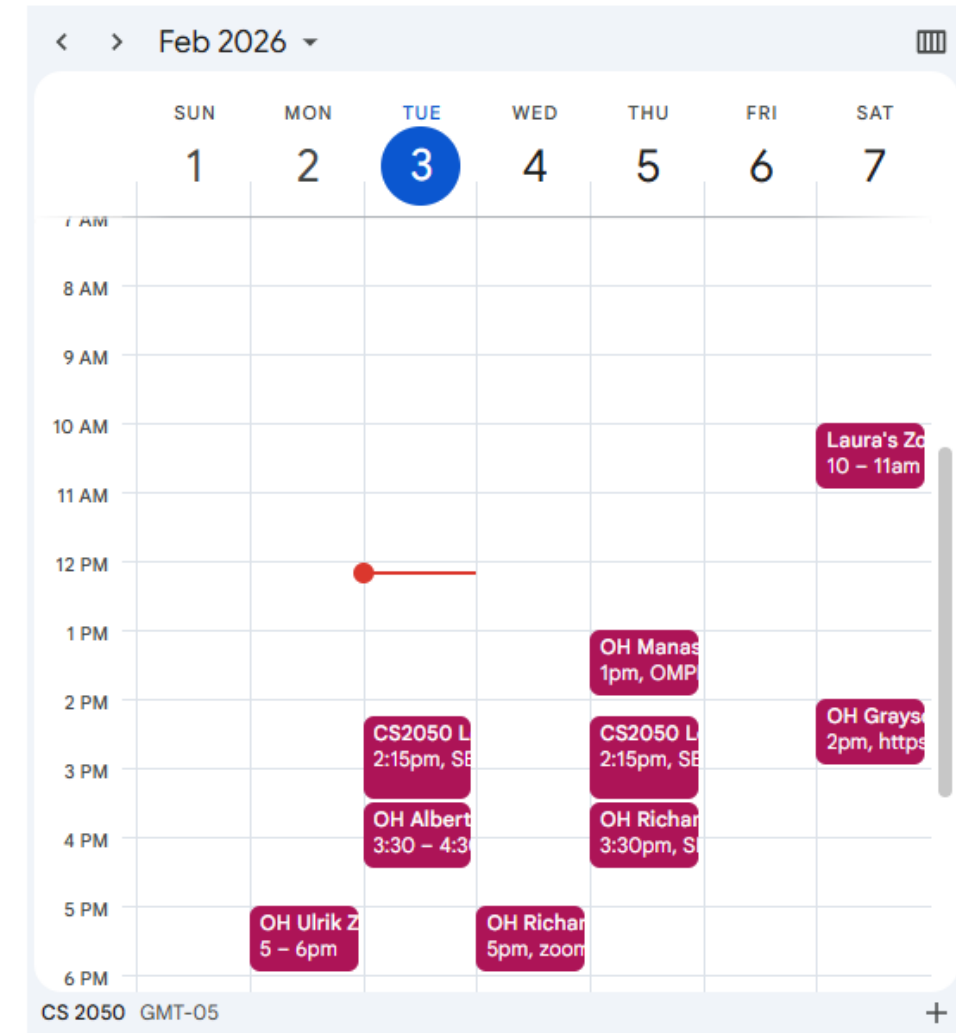


2025-2026 Spring

- Home
- Announcements
- Course Calendar
- Q & A
- Zoom
- DCE Class Recordings
- Quizzes
- Gradescope
- People
- Grades
- Files

S

Course Google Calendar



WHAT HAPPENS WHEN YOU RUN A PROGRAM?

WHAT HAPPENS WHEN YOU RUN A PROGRAM?

Assume we have this simple high-level C program

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     printf("hello, world!\n");
6     return 0;
```

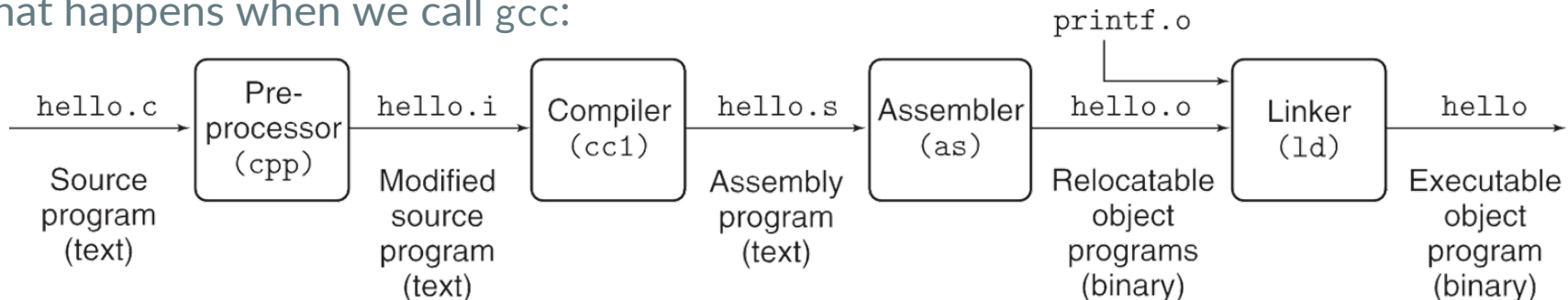
(high-level because humans can read and understand it)

In order to run this program, the high-level code must be *translated* into a sequence of low-level *machine-language instructions*

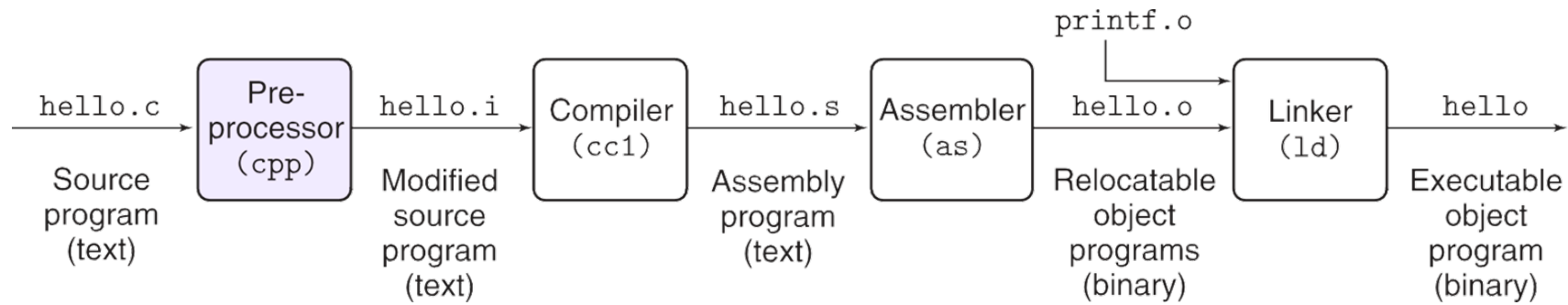
A *compiler* translates the source code into machine instructions which are packaged into an *executable object program* (or *binary*). For the code above we would execute

```
$ gcc -o hello hello.c
```

What happens when we call gcc:



WHAT HAPPENS WHEN YOU RUN A PROGRAM?

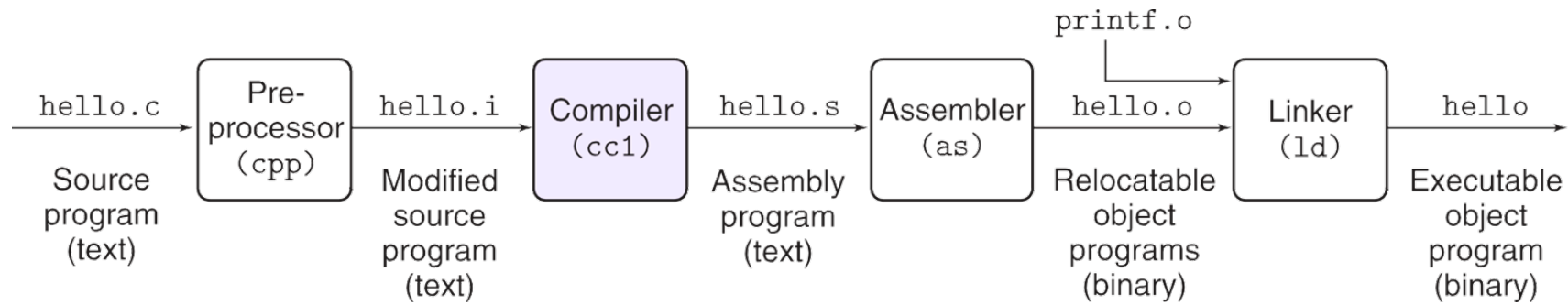


Preprocessing phase:

The *preprocessor* modifies the original program depending on directives that start with "#". Comments will be removed as well. For example, the `#include <stdio.h>` header in our code would be expanded and replaced with code in this phase. You can investigate this stage with

```
$ gcc -E hello.c >hello.i
```

WHAT HAPPENS WHEN YOU RUN A PROGRAM?



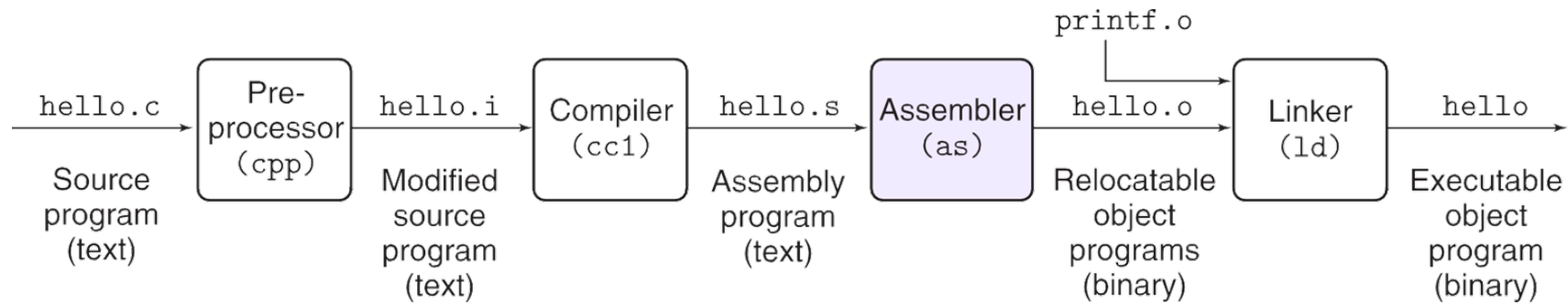
Compilation phase:

The *compiler* translates the pre-processed file into an *assembly-language* file. Assembly language is useful because it allows us to *inspect the generated machine instructions in a human readable form*. This is important when we optimize. Moreover, it provides a common output language, i.e. C/C++ and Fortran would look the same on this level.

```
1 main:
2   subq $8, %rsp
3   leaq .LC0(%rip), %rdi
4   call puts@PLT
5   xorl %eax, %eax
6   addq $8, %rsp
7   ret
```

Example assembly code for the `hello.c` program. Lines 2-7 are each machine's instructions.

WHAT HAPPENS WHEN YOU RUN A PROGRAM?



Assembly phase:

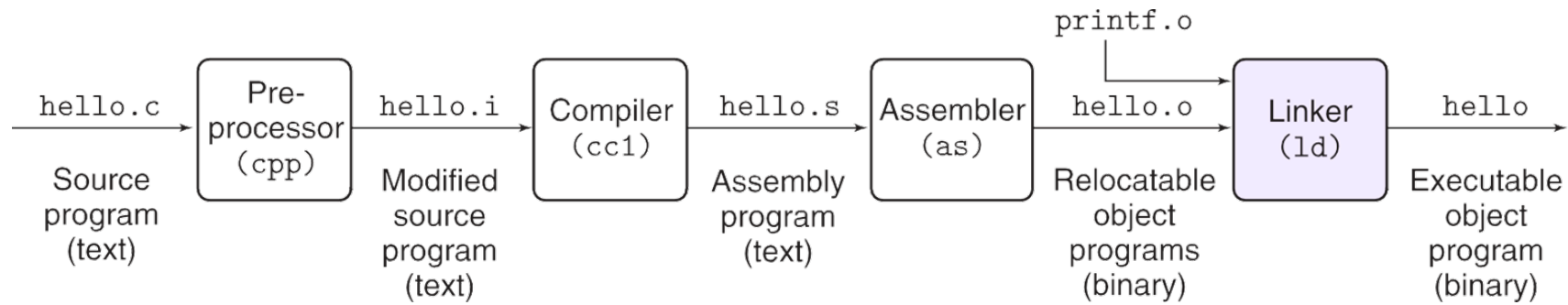
The *assembler* translates assembly code input into machine-language instructions and packages them into a form known as a *relocatable object program* which usually have a *.o file suffix. This output can be generated with

```
$ gcc -c hello.c
```

When we *disassemble* hello.o we find that our main function is encoded by 23 bytes and we have a much harder time to figure out what this machine code does (without the assembly on the right). We do not need to understand this output for CS 2050

```
$ objdump -d hello.o
0000000000000000 <main>:
   0: 48 83 ec 08          sub    $0x8,%rsp
   4: 48 8d 3d 00 00 00 00 lea     0x0(%rip),%rdi      # b <main+0xb>
   b: e8 00 00 00 00      call   10 <main+0x10>
  10: 31 c0                xor    %eax,%eax
  12: 48 83 c4 08          add    $0x8,%rsp
  16: c3                  ret
```


WHAT HAPPENS WHEN YOU RUN A PROGRAM?



Linking phase:

Note that our program calls `printf` which is part of the *standard C library* provided by the compiler (similar to the OpenMP library that we will use later in the class). The `printf` function resides in a separate `printf.o` file which must be *linked* to our `hello.o` file. The result is the **executable** `hello` file that can be loaded into memory and executed by the system. Assuming the `printf.o` file was in the same directory as `hello.o` we could run:

```
$ gcc -o hello hello.o printf.o
```

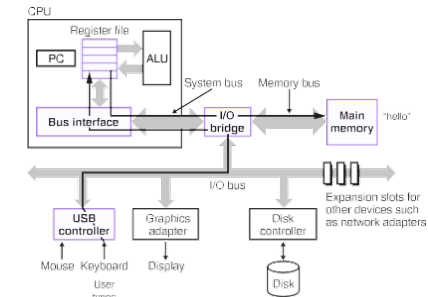
(In practice `printf.o` is part of the standard library and the compiler will find it without us explicitly specifying it.)

WHAT HAPPENS WHEN YOU RUN A PROGRAM?

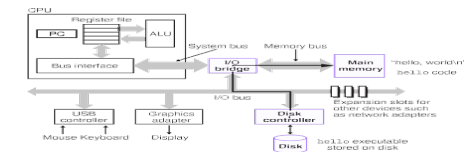
- When we execute ...

```
$ ./hello
hello, world!
```

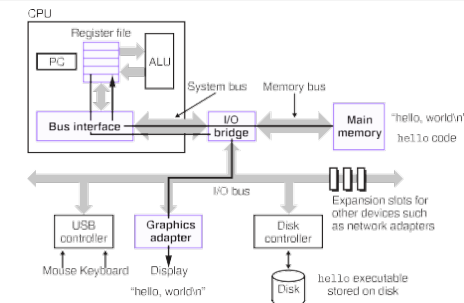
We type the `./hello` characters in the shell which are read one by one into a register and stored in main memory. When we hit enter the shell knows we are done and execution of the program begins.



The shell then loads the executable `hello` file by executing the instructions that copy the code and data in the `hello` object file from *disk* to *main memory*. The data includes the string "hello, world!\n" which is encoded in the executable.



Once the code and data in the `hello` object file are loaded in memory, the processor begins to execute the instructions in the main function. These instructions copy the `hello, world!\n` string (characters) from memory to the register file and from there to the display device.



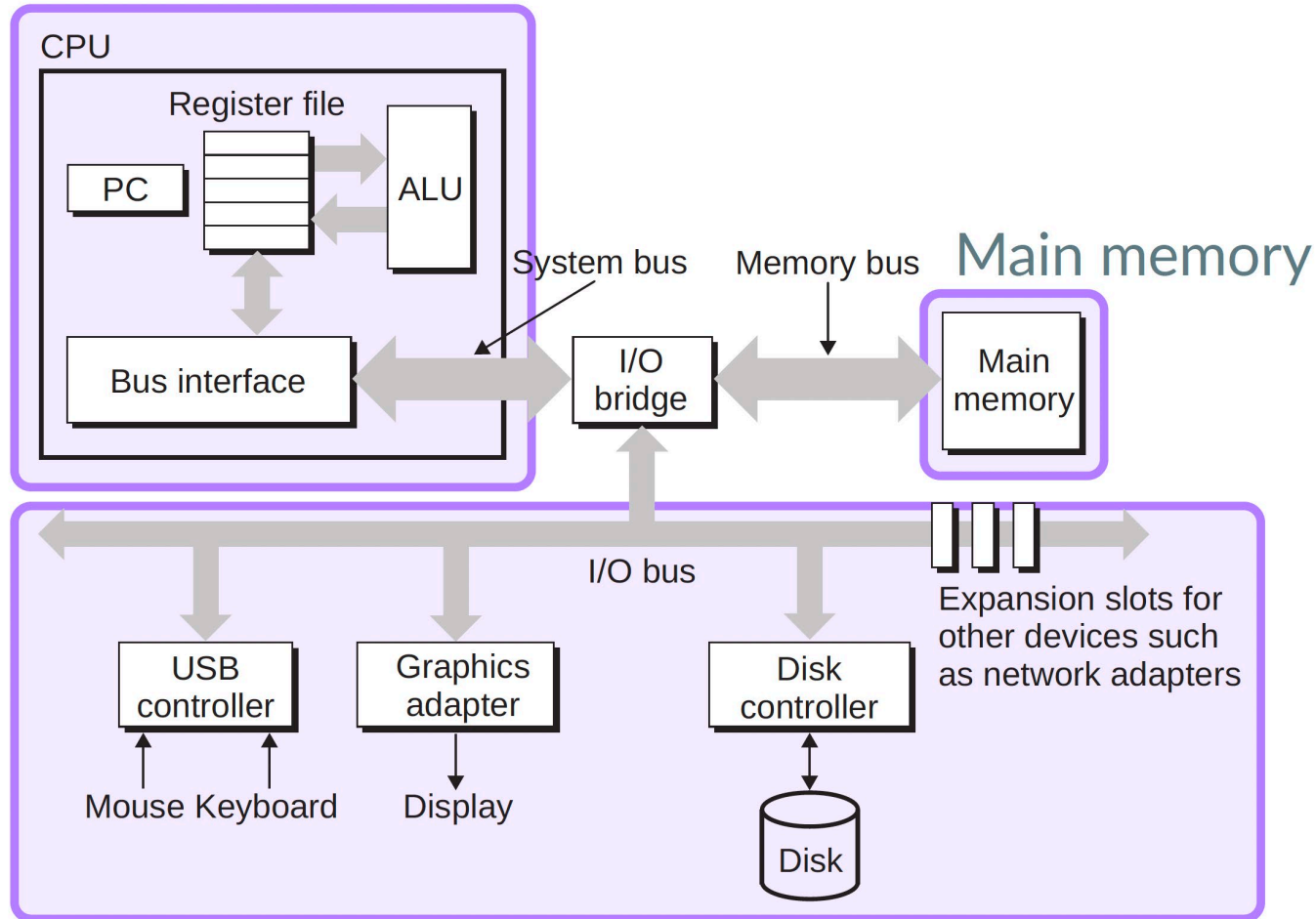
COMPUTER ARCHITECTURE AND PERFORMANCE IMPLICATIONS

WHY LEARN ABOUT HARDWARE?

- Performance depends on both algorithms and hardware
- Hardware details change; performance principles do not
- Goal of this section:
 - Understand *why* programs run fast or slow
 - Develop mental models for hardware that make you a better programmer

TYPICAL COMPUTER SYSTEM

Processor



I/O devices

Main components:

- Main memory (temporary storage)
- Processor (central processing unit)
- Buses (electrical conduits)
- I/O devices (input/output)

CORE COMPONENTS: MAIN MEMORY

Main memory is a temporary storage device

It holds both, a program and data it manipulates while the processor is executing the program

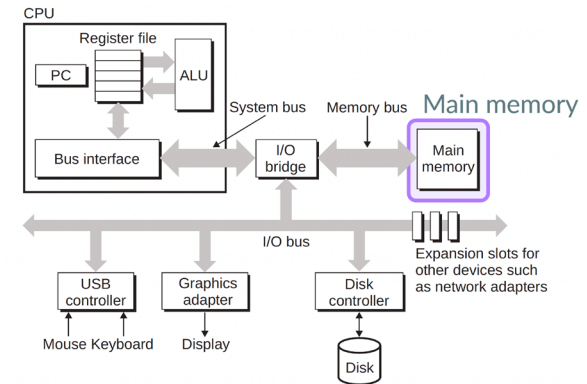
Physically, main memory consists of a collection of dynamic random access memory (DRAM)

"Dynamic" means that the memory cells must be refreshed periodically (DRAM has a frequency, usually MHz)

Logically, memory is organized as a linear array of bytes, each with its own unique memory address, starting at zero

You can think of a memory address as the index into the byte array.
Each byte has one address:

- 32-bit system: 4294967296 addresses → maximum 4GB DRAM
- 64-bit system: 18446744073709551616 addresses → maximum 16EB DRAM



CORE COMPONENTS: PROCESSOR

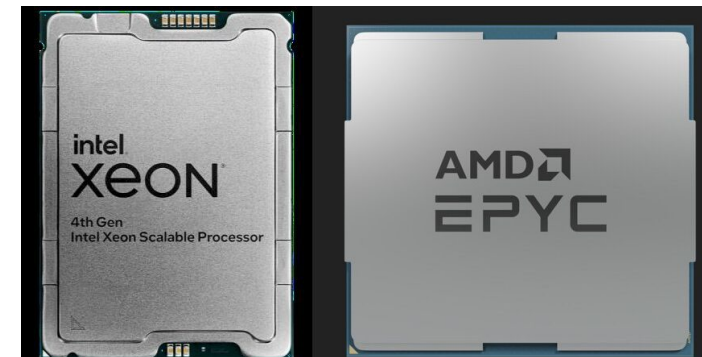
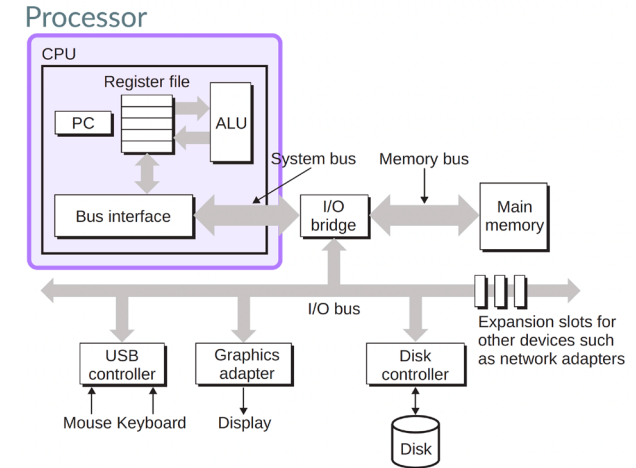
The central processing unit (CPU) or simply processor is the workhorse that interprets (executes) instructions stored in main memory

At its core is a word-size storage device (a register) called the program counter (PC). At any point in time the PC points at (contains the address of) some machine instruction in main memory

From power on until power off the processor repeatedly executes the instruction pointed at by the PC and updates it to point to the next instruction

The processor operates with a (simple) instruction model, defined by its instruction set architecture (ISA). Two common models are RISC (reduced instruction set computer) and CISC (complex instruction set computer).

Instructions execute in strict sequence and executing a single instruction involves a series of steps

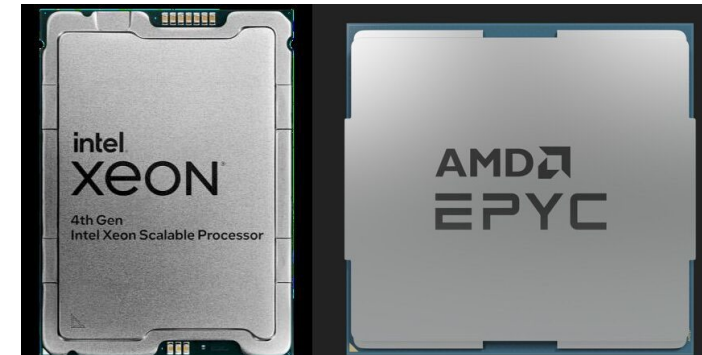
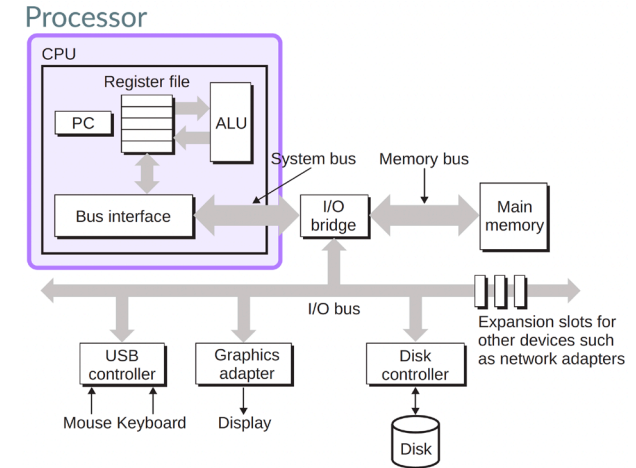


CORE COMPONENTS: PROCESSOR

There are only a few of these simple operations and they revolve around main memory, the register file and the arithmetic/logic unit (**ALU**)

The register file is a small storage device that consists of a collection of word-sized registers

The ALU computes new data and address values.



CORE COMPONENTS: PROCESSOR

Some simple operations the CPU might carry out:

Load:

Copy a byte or a word from main memory into a register, overwriting the previous contents of the register

Store:

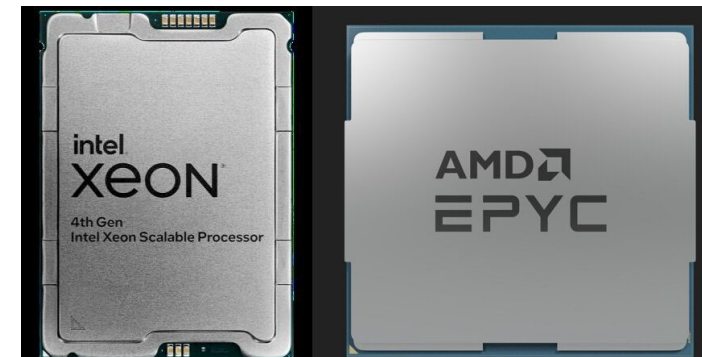
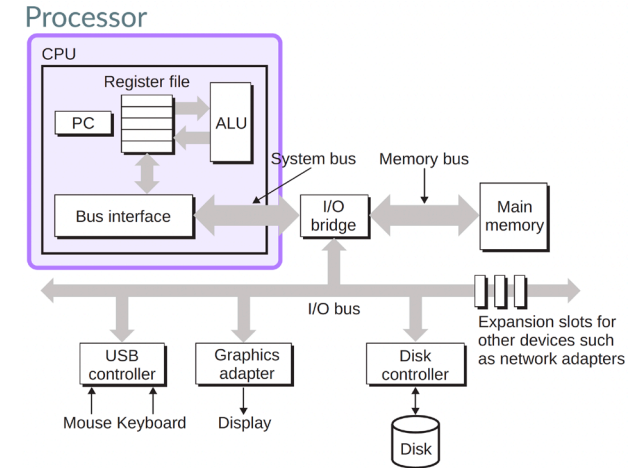
Copy a byte or a word from a register to an address in main memory, overwriting the previous contents at that base address

Move:

Move the value in one register to another (no access to main memory)

Compare:

Compare the value in a register to zero



CORE COMPONENTS: BUSES

Are a collection of electrical circuits that carry bytes of information back and forth between components

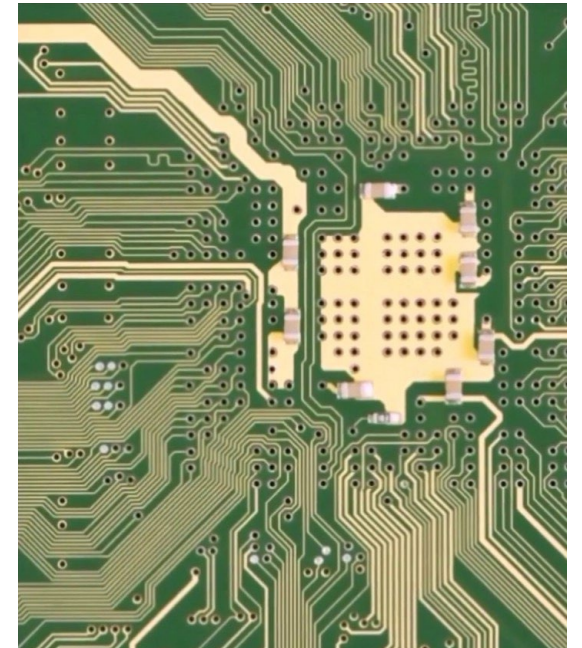
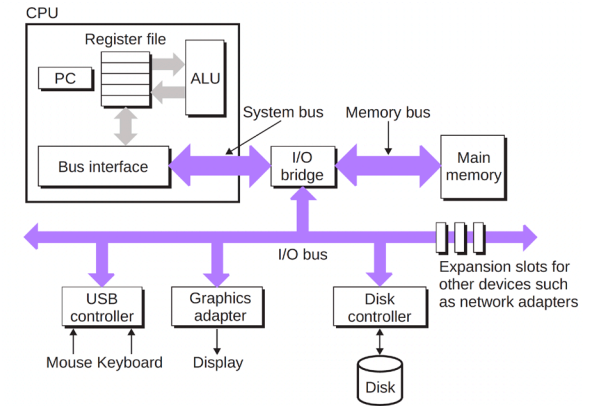
Typically, the transferred chunks of bytes are known as **words**

The number of **bytes in a word** is called the **word size** and is a fundamental system parameter

Most machines today have word sizes of either 4 bytes (32 bits) or 8 bytes (64 bits).

The maximum rate at which byte chunks can be transferred is called **bandwidth**

Typical bandwidth on a recent CPUs is ~400 GB/s and much larger on a GPU. Why?



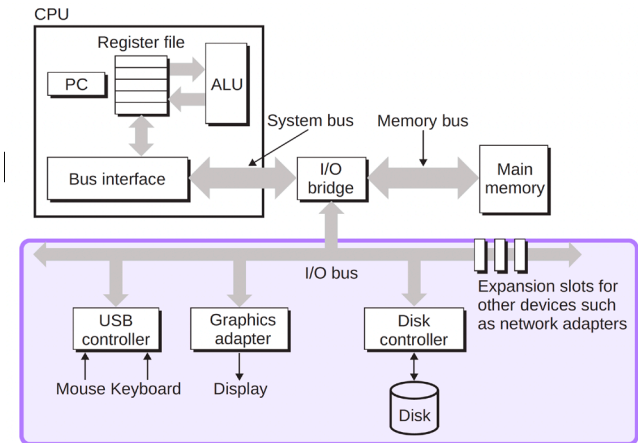
Bus system on a motherboard

CORE COMPONENTS: I/O DEVICES

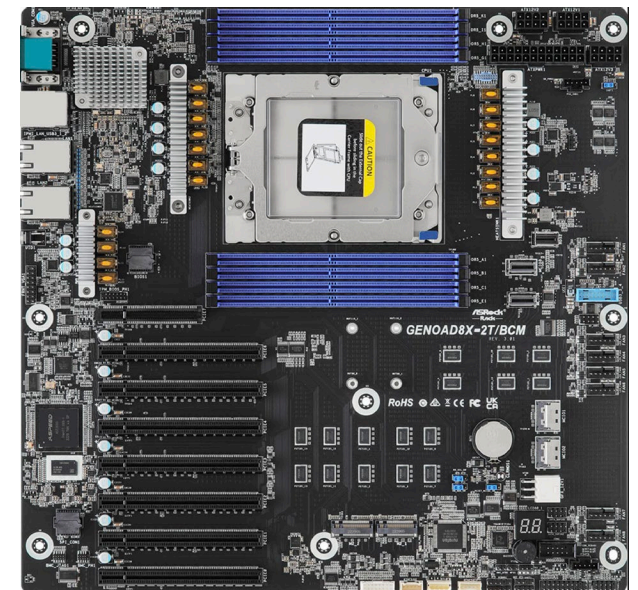
I/O (Input/Output) devices are the connection to the outer world. Examples: keyboard, mouse, display, disk drive, printer, GPU (rendering, PCIe)

I/O devices are connected by controllers (with chip set) or adapters (plugin card, e.g. GPU)

Purpose of controller or adapter is simply to transfer information between I/O bus and I/O device

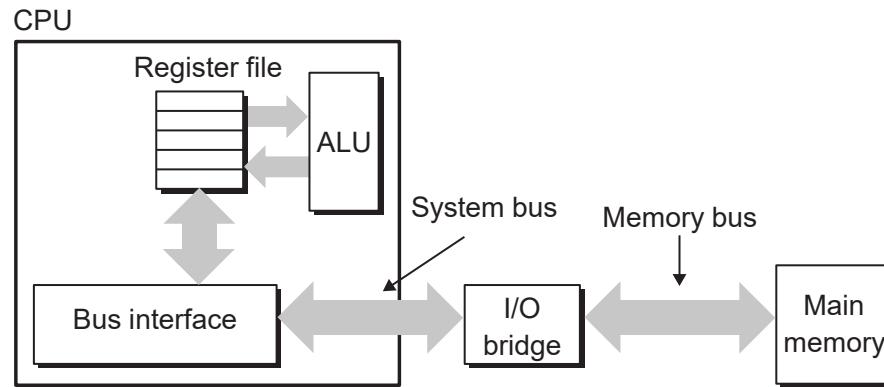


I/O devices



PCIe adapter (right) and a server motherboard with expansion slots (left)

VON NEUMANN ARCHITECTURE



von Neumann architecture



John von Neumann

- Classic von Neumann architecture consists of:
(1) main memory, (2) processor, and (3) interconnect
- Main memory stores both instructions and data, which are transferred through the interconnect (bus)

WHAT'S MISSING FROM THIS ARCHITECTURE?

- Many auxiliary (e.g. I/O) devices
- Overly simplified memory model

VON NEUMANN ARCHITECTURE

von Neumann bottleneck: separation of processor and memory



Warehouse (main memory)



Traffic (interconnect/bus)



Factory (processor)

The interconnect determines the rate at which instructions and data can be accessed. The traffic jam above is problematic because the folks in the factory will have to wait.

Latency: the **response time** for information to arrive measured from its initial request. Latency is often related to physical parameters the time to fill the pipe is proportional to its dimension.

Bandwidth: the throughput of a subsystem. Naively the inverse of latency. Modern subsystems can overlap multiple requests to increase bandwidth!

More generally defined as the **rate at which requests can be satisfied**.

LIBRARY PARADIGM

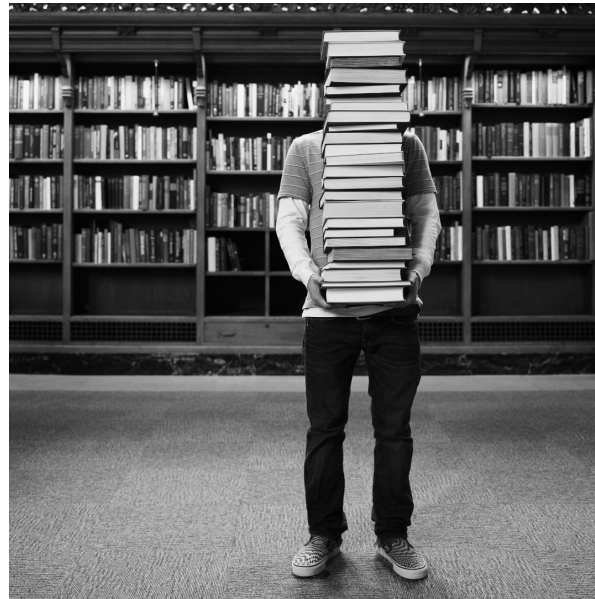
Assume you are writing a paper

You go to the local library to work undisturbed

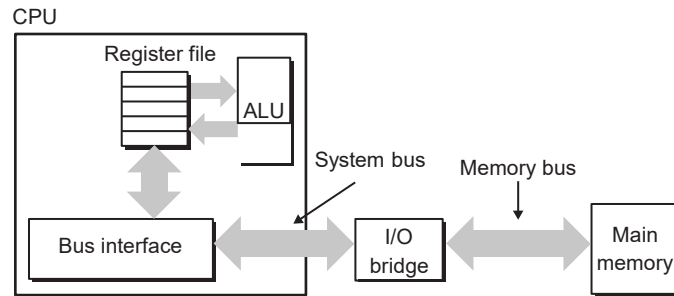
How do you organize yourself?



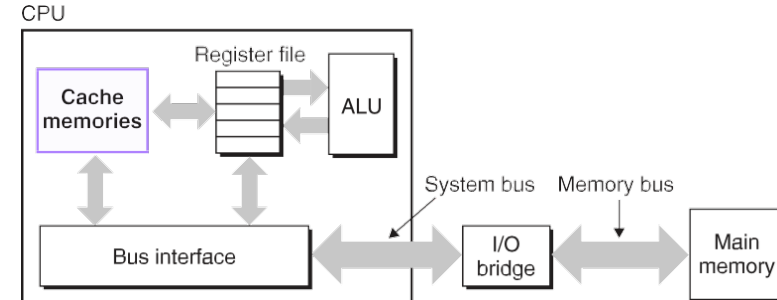
Do you constantly race back and forth in the library shelves?



CACHE MEMORIES



Classic von Neumann architecture



von Neumann architecture *with on-chip caches*

Caches are small *on-chip memories*, much like the space you have on your desk where you put your books in the library example.

CACHE MEMORIES

Caches are small *on-chip memories*, much like the space you have on your desk where you put your books in the library example before.

In most architectures there are *three* levels of caches:

- **L1:** Smallest and closest cache to the ALU. Often exclusive to core
- **L2:** Intermediate cache after L1. Often 256-512kB in size but can be larger. Often shared among a few cores.
- **L3:** Last level cache (LLC) closest to DRAM. Size varies on type of CPU and may be in the range of 4-64MB. Typically shared among cores.

CACHE MEMORIES

Machine (3442MB total)

Package L#0

NUMANode L#0 P#0 (3442MB)

L2 (4096KB)

L2 (16MB)

L1d (64KB)

L1d (64KB)

L1d (64KB)

L1d (64KB)

L1d (128KB)

L1d (128KB)

L1d (128KB)

L1d (128KB)

L1i (128KB)

L1i (128KB)

L1i (128KB)

L1i (128KB)

L1i (192KB)

L1i (192KB)

L1i (192KB)

L1i (192KB)

Core L#0

Core L#1

Core L#2

Core L#3

Core L#4

Core L#5

Core L#6

Core L#7

PU L#0
P#0

PU L#1
P#1

PU L#2
P#2

PU L#3
P#3

PU L#4
P#4

PU L#5
P#5

PU L#6
P#6

PU L#7
P#7

L2 (16MB)

L1d (128KB)

L1d (128KB)

L1d (128KB)

L1d (128KB)

L1i (192KB)

L1i (192KB)

L1i (192KB)

L1i (192KB)

Core L#8

Core L#9

Core L#10

Core L#11

PU L#8
P#8

PU L#9
P#9

PU L#10
P#10

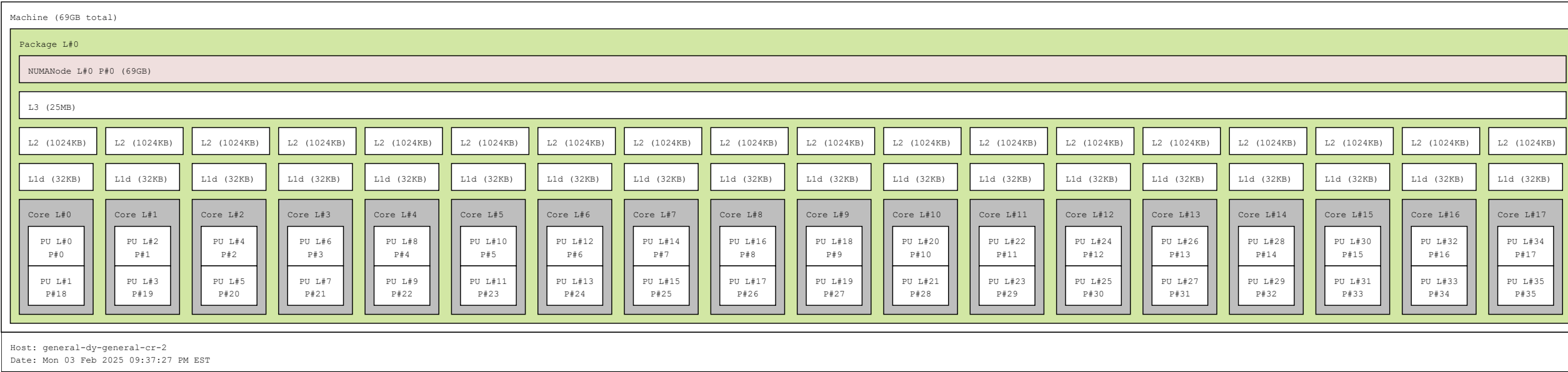
PU L#11
P#11

Host: HUIT-SEAS-huitadmins-MacBook-Pro

Date: Mon Feb 3 21:49:16 2025

Apple M2 Pro

CACHE MEMORIES



Intel(R) Xeon(R) Platinum 8124M

CACHE MEMORIES

There are typically **two** level 1 caches:

- **L1d**: Level 1 *data* cache (exclusive for data)
- **L1i**: Level 1 *instruction* cache (exclusive for instructions).

There is a separate L1 instruction cache because of different access patterns and pipelines.

The level 2 and 3 caches are *unified*. They can cache either *data* or *instructions*.

Latencies associated with these cache levels:

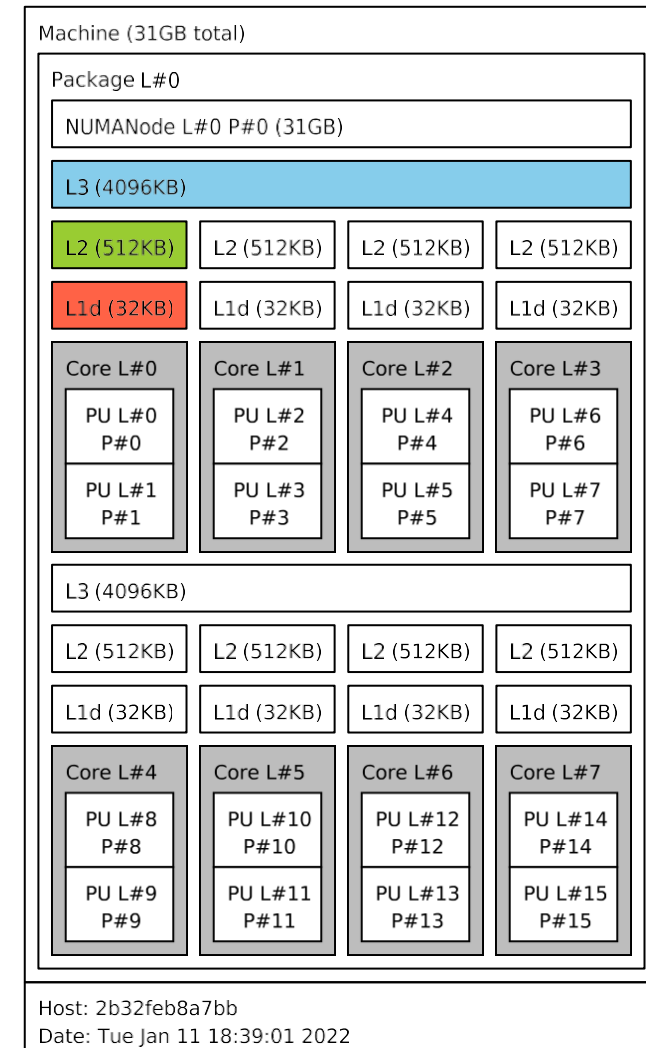
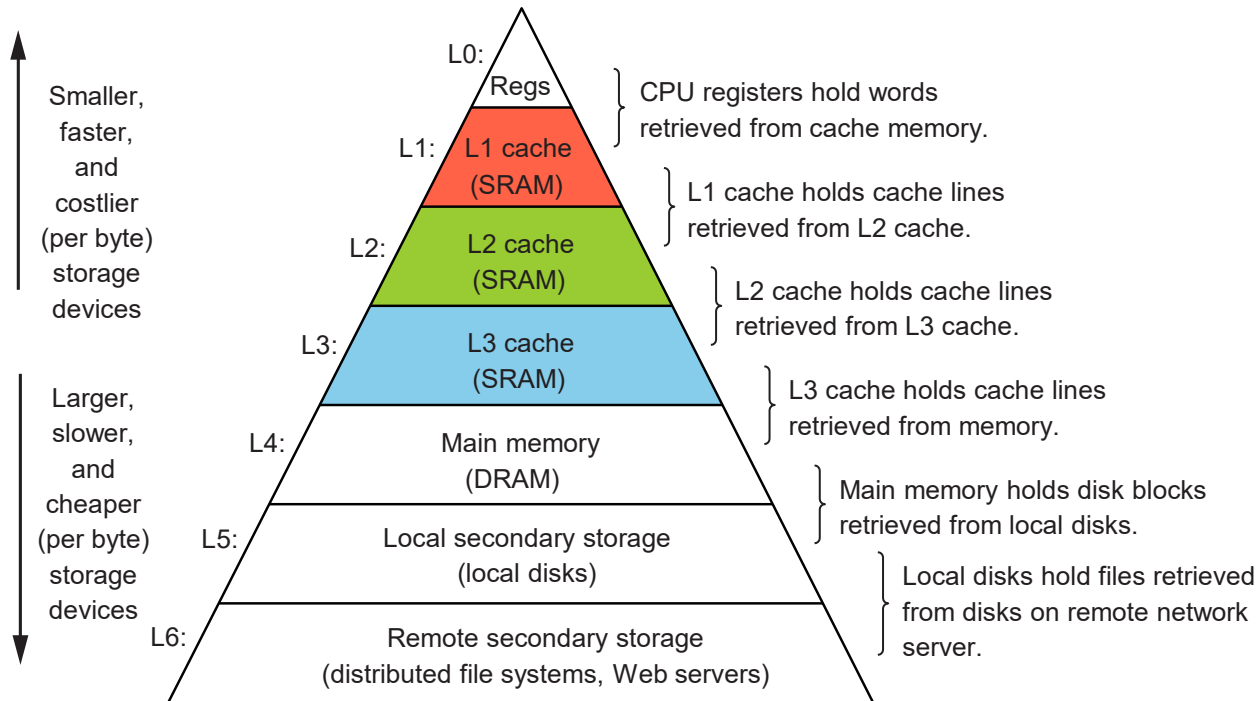
Type	Where cached	Latency (cycles)	Managed by
CPU registers	On-chip CPU registers	0	Compiler
L1 cache	On-chip L1 cache	4	Hardware
L2 cache	On-chip L2 cache	10	Hardware
L3 cache	On-chip L3 cache	50	Hardware

MORE LATENCY NUMBERS

Type	Where cached	Latency (cycles)	Managed by
CPU registers	On-chip CPU registers	0	Compiler
L1 cache	On-chip L1 cache	4	Hardware
L2 cache	On-chip L2 cache	10	Hardware
L3 cache	On-chip L3 cache	50	Hardware
Virtual memory	Main memory (DRAM)	200	Hardware + OS
Disk cache	Disk controller	100'000	Controller firmware
Browser cache	Local disk	10'000'000	Web browser
Web cache	Remote server disks	1'000'000'000	Web proxy server

CACHE MEMORIES

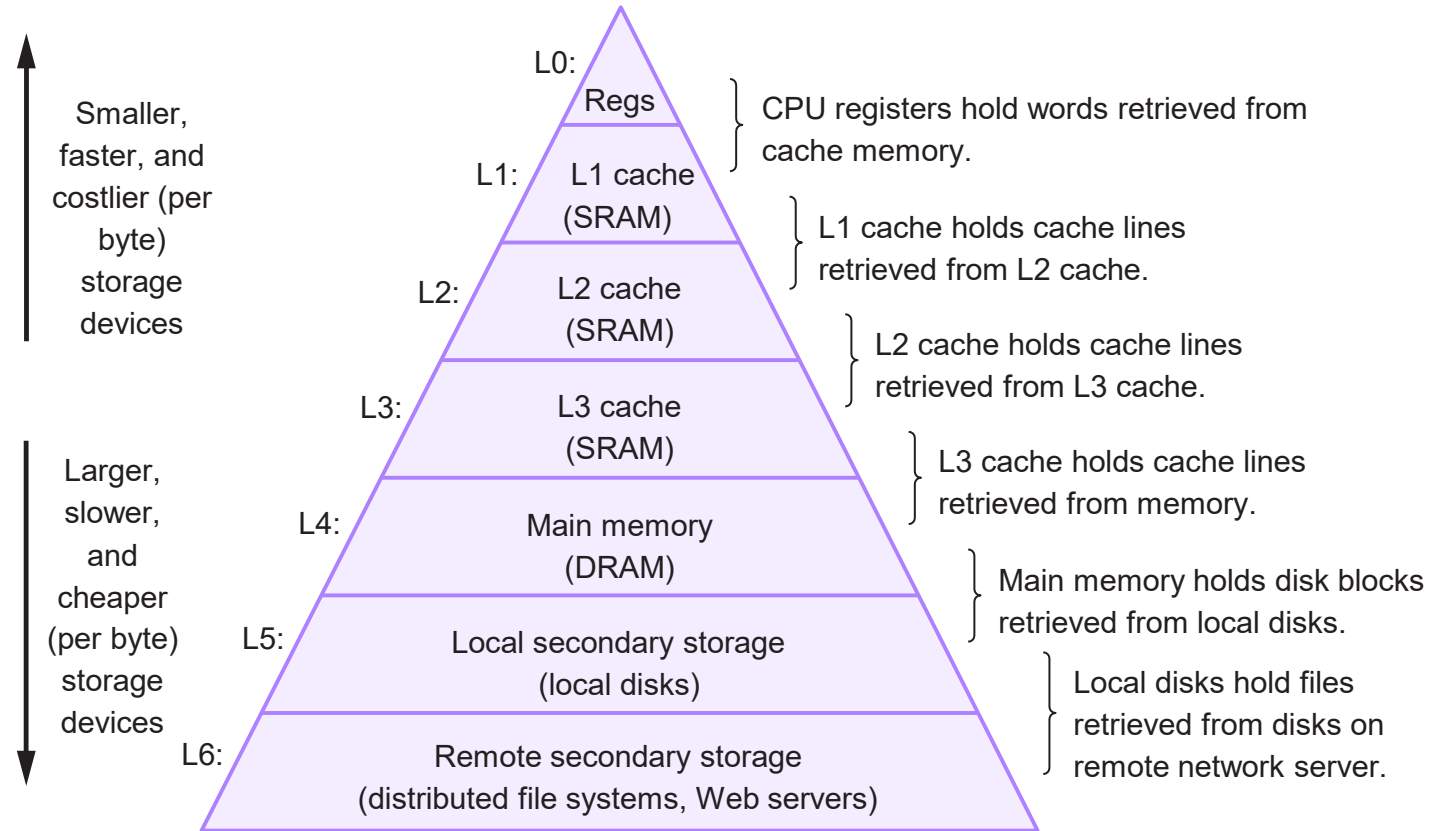
Why are there multiple levels of cache and not just one big cache?



CACHE MEMORIES

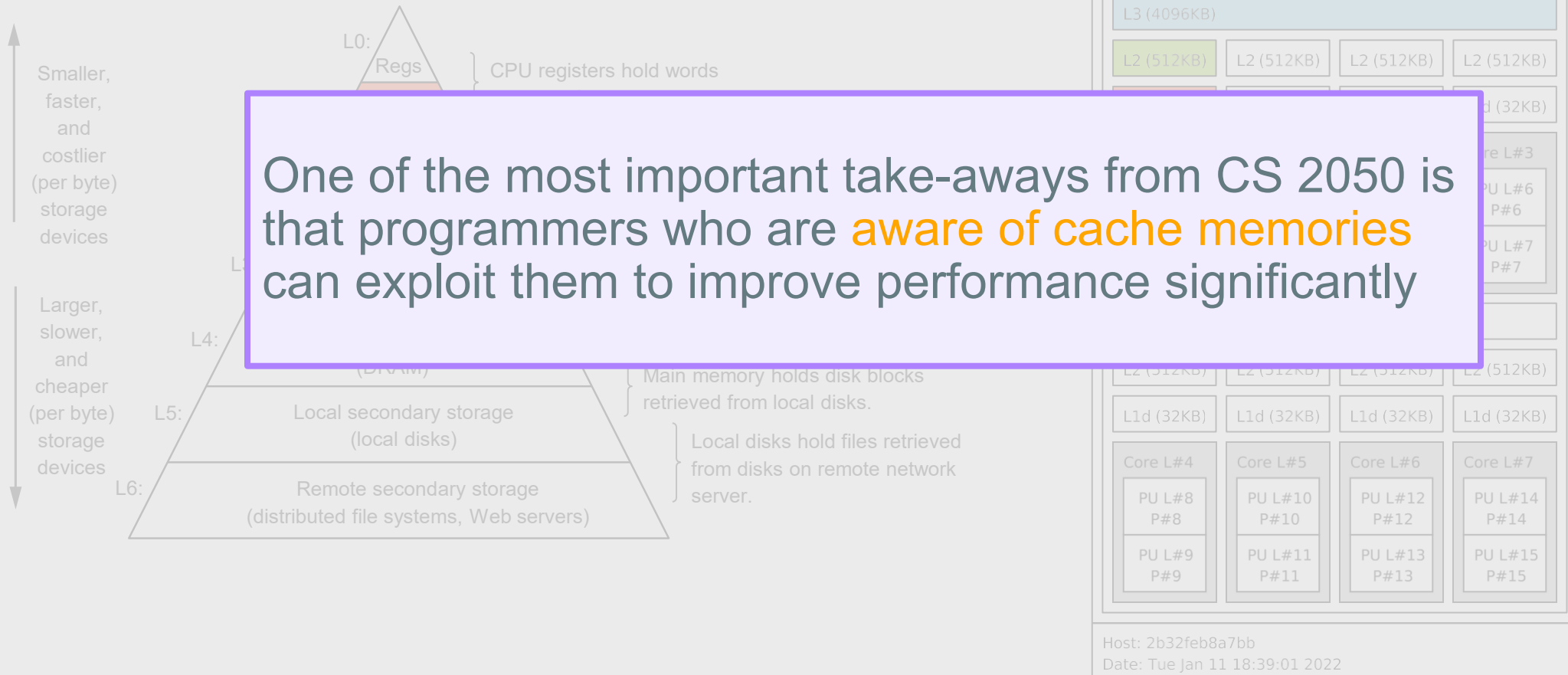
Since 1985, the cost per megabyte DRAM has decreased by a factor of 44'000(!), its access time has improved only by a factor of 10

The cost of SRAM is about 50-100 times higher than DRAM because of the manufacturing process as well as one SRAM cells requires 6 transistors while DRAM requires 1 (and a capacitor). SRAM is much more power hungry than DRAM.



CACHE MEMORIES

So why are there multiple level caches and not just one big cache?



THE PRINCIPLE OF LOCALITY

High performing code makes good use of *locality*

A program with *good locality* references data items that are *spatially near* other recently referenced items or were recently referenced themselves (*temporal locality*)

Every memory level in a computer system acts as a cache designed to exploit locality

LOCALITY

A variable that is referenced frequently in a short time interval is said to have good *temporal locality*

Given the array:

```
int v[N];    // N elements in the array
```

with the following memory layout:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----

where the numbers in the blocks indicate the array indices.

Consider now the following code:

```
int sum = 0;
for (int i = 0; i < N; ++i) {
    sum += v[i];
}
```

Is this pattern sensible?

LOCALITY



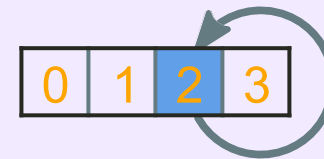
Consider these nested loops:

```
for (int loop = 0; loop < 10; ++loop) {  
    for (int i = 0; i < N; ++i) {  
        ... = ... v[i] ...;  
    }  
}
```

```
for (int i = 0; i < N; ++i) {  
    for (int loop = 0; loop < 10; ++loop) {  
        ... = ... v[i] ...;  
    }  
}
```

Is one pattern more sensible?

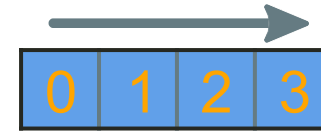
In the library paradigm you took the book to your work desk because you will use it frequently



SPATIAL LOCALITY

Recall this loop:

```
int sum = 0;
for (int i = 0; i < N; ++i) {
    sum += v[i];
}
```



The variable `v[i]` exhibits good spatial locality because every element `i` access is immediately followed by `i-1` (related books are next to each other in the shelves)

A sequential access pattern like in the loop above is called stride-1 reference pattern (sometimes sequential reference pattern or unit stride pattern)

STRIDED MEMORY ACCESS

Algorithms may not always allow for unit stride access patterns

A *stride- k reference pattern* visits every k -th element of a contiguous vector

The larger k , the smaller the spatial locality (cache size is limited!)

Stride-4 reference pattern:



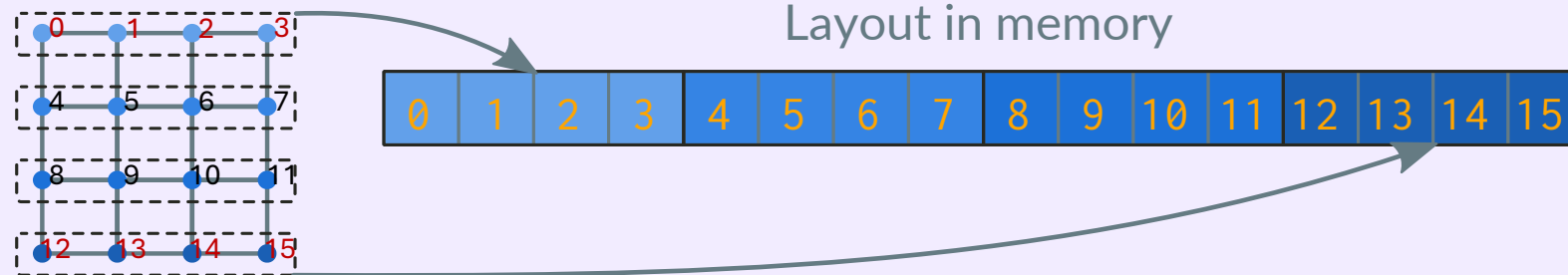
HIGHER DIMENSIONAL ARRAYS

All memory in a computer is a *contiguous* vector of bytes

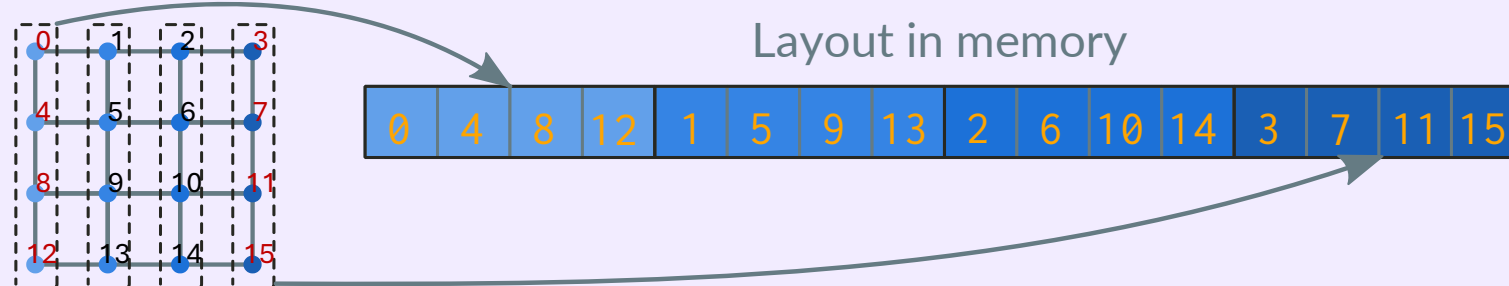
How are higher-dimensional arrays represented in memory?

Two main distinctions: *row-major* and *column-major* order:

Row-major: used in C or C++



Column-major: used in Fortran



Analogous patterns for higher dimensions

EXAMPLES OF LOCALITY

Which of the two codes below exhibits *better* spatial locality for a *row-major order* (referencing elements that are nearby in memory)?

Code A:

```
float A[N][N];
float sum = 0.0;

for (int i = 0; i < N; ++i) {
    for (int j = 0; j < N; ++j) {
        sum += A[i][j]
    }
}
```

Code B:

```
float A[N][N];
float sum = 0.0;

for (int j = 0; j < N; ++j) {
    for (int i = 0; i < N; ++i) {
        sum += A[i][j]
    }
}
```

Note that A can be written with a single loop:

```
float *p = &A[0][0];
float sum = 0.0;

for (int i = 0; i < N * N; ++i) {
    sum += p[i]; // element access is identical
}
```

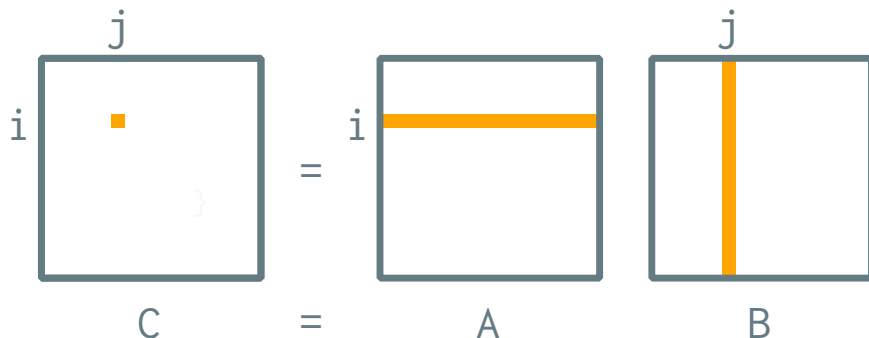
EXAMPLES OF LOCALITY

The matrix-matrix multiplication $C = AB$ can be computed in different ways. Assume all matrices are stored in row-major order and there is no cache (for now).

How does the temporal and spatial locality differ from the following two implementations:

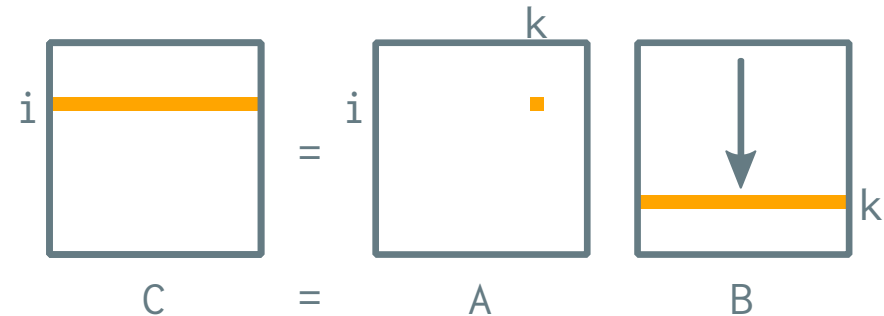
Implementation 1

```
for (int i = 0; i < N; ++i) {  
    for (int j = 0; j < N; ++j) {  
        for (int k = 0; k < N; ++k) {  
            C[i][j] += A[i][k] * B[k][j]  
        }  
    }  
}
```



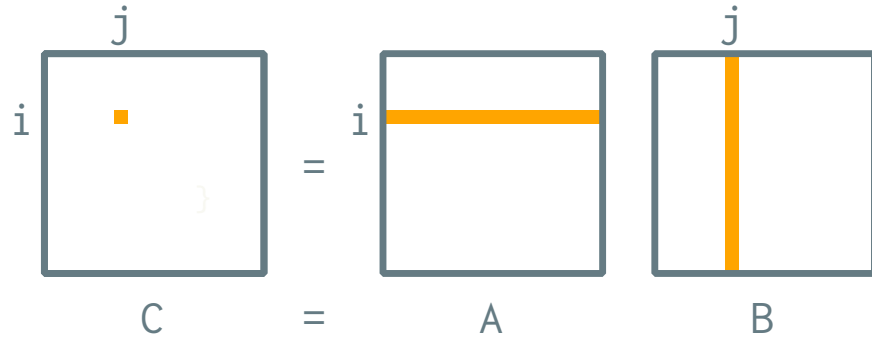
Implementation 2

```
for (int i = 0; i < N; ++i) {  
    for (int k = 0; k < N; ++k) {  
        for (int j = 0; j < N; ++j) {  
            C[i][j] += A[i][k] * B[k][j]  
        }  
    }  
}
```

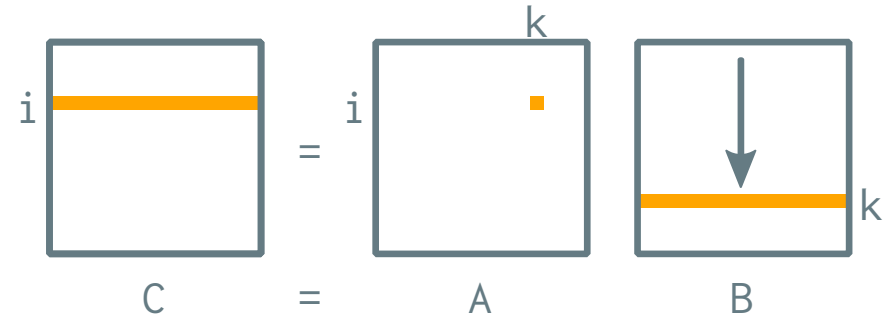


EXAMPLES OF LOCALITY

Implementation 1



Implementation 2



$C[i][j]$

In the first implementation $C[i][j]$ is invariant and has good temporal locality (will be stored in register). In the second implementation we must load $C[i][j]$ in each inner iteration (this implies more store operations).

$A[i][k]$

Both implementations address elements in A by rows (good spatial locality). For the second implementation $A[i][k]$ is invariant w/r/t to inner-most loop (good temporal locality).

$B[k][j]$

The magic difference is the access pattern for B. The second implementation exhibits better spatial locality than the first implementation. Especially when caches are involved, the access pattern of the first implementation is bad because we must load *cache lines*.

CACHES: TERMINOLOGY

Cache block/line

Fixed size packet that moves back and forth between cache levels or main memory. It is a container that stores a block as well as other information such as the valid and tag bits (not important for us).

Cache hit

Data request by the processor that is *found in level k* . In that case the data is read directly from the level k cache which is faster than level $k + 1$.

Cache miss

If the *data is not found*, then it is a *cache miss*. In this case a new cache line must be *fetched* from the level $k + 1$ cache. This line must *evict* an existing line in level k (the *victim*) according to a replacement policy. Possible policies are *random replacement*, *least recently used (LRU)* or *least frequently used (LFU)* policies for example.

Hit/miss rate

Fraction of memory accesses that are cache hits (or misses) in the level k cache.

Miss penalty

Time to replace a block in level k with the corresponding block from level $k + 1$.

CACHES

When you access a data element in your data structure, multiple coalesced elements are loaded at once, not just one.

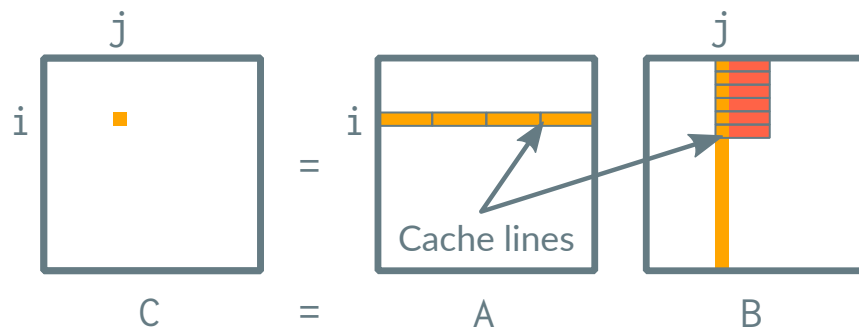
Library paradigm: you take whole books to your desk, not just single pages.

A cache line is usually tens of bytes. On most Intel and AMD systems it is **64 byte**
→ n-byte cache lines are aligned at n-byte boundaries!

For example, 64 bytes are:

- 16 elements of 4-byte types (int or float)
- 8 elements of 8-byte types (long or double)

Recall: implementation 1 of previous matrix-matrix multiplication example:



- Each access $B[k][j]$ loads a cache line
- From the, say, 8 loaded elements, we *only* use 1!
- ***This is inefficient and wastes a lot of bandwidth!***

EXAMPLE: CACHE MISSES IN DOUBLY NESTED LOOP

Assume a fully-associative cache can hold 4 cache blocks. Each cache block can hold 4 single precision floating point numbers such that the cache can hold 16 floats in total. Determine the number of any *cache misses* and the *hit rate* in the following C++ codes:

Case A:

```
float A[8][8]; // 64 float numbers
float sum = 0.0;

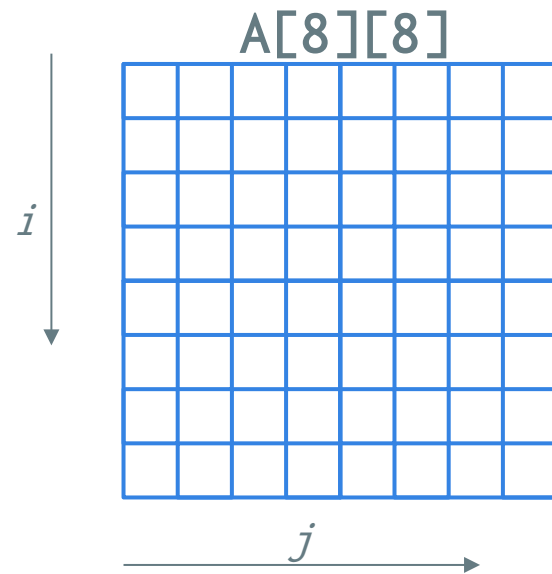
for (int j = 0; j < 8; ++j) {
    for (int i = 0; i < 8; ++i) {
        sum += A[i][j];
    }
}
```

Case B:

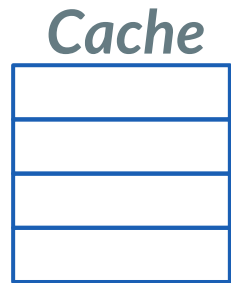
```
float A[8][8]; // 64 float numbers
float sum = 0.0;

for (int i = 0; i < 8; ++i) {
    for (int j = 0; j < 8; ++j) {
        sum += A[i][j];
    }
}
```

CASE A

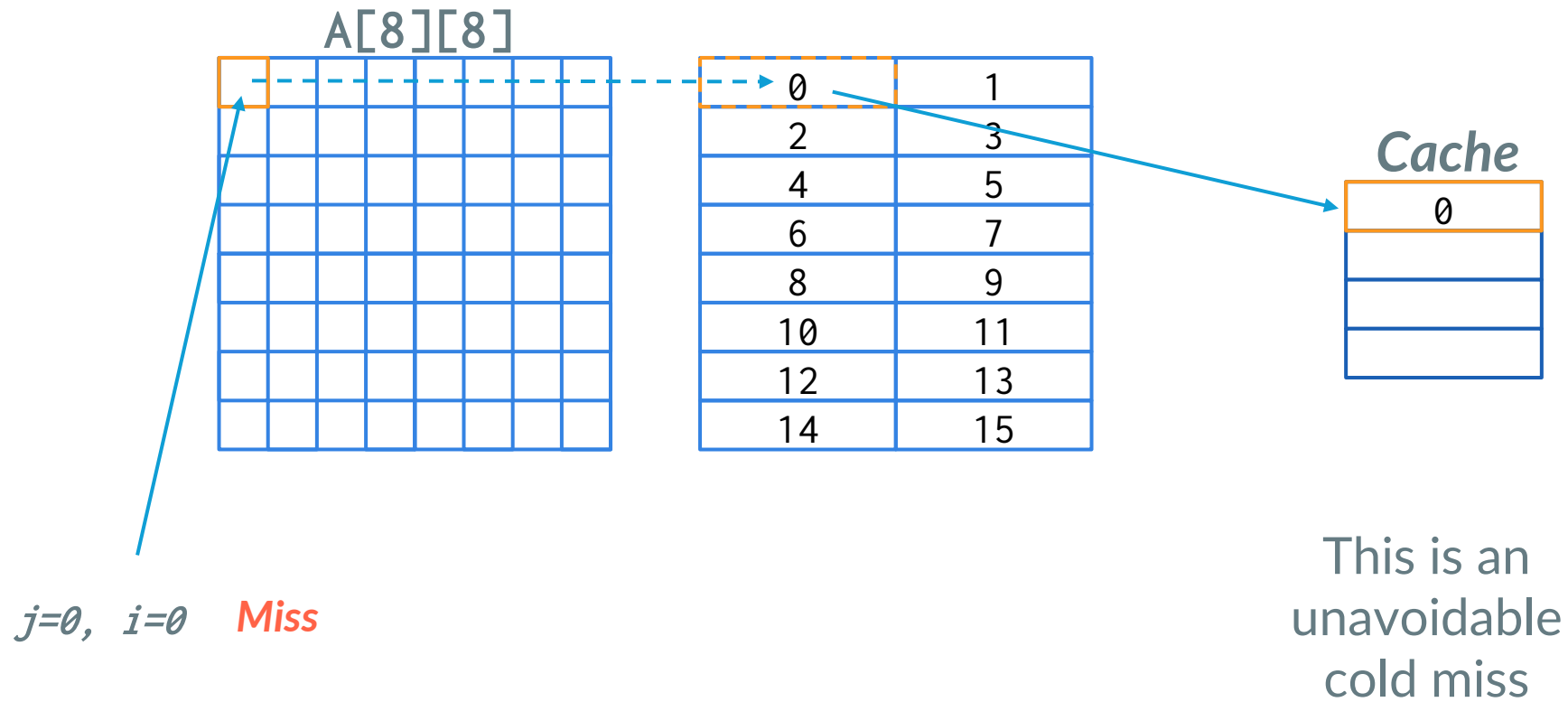


0	1
2	3
4	5
6	7
8	9
10	11
12	13
14	15

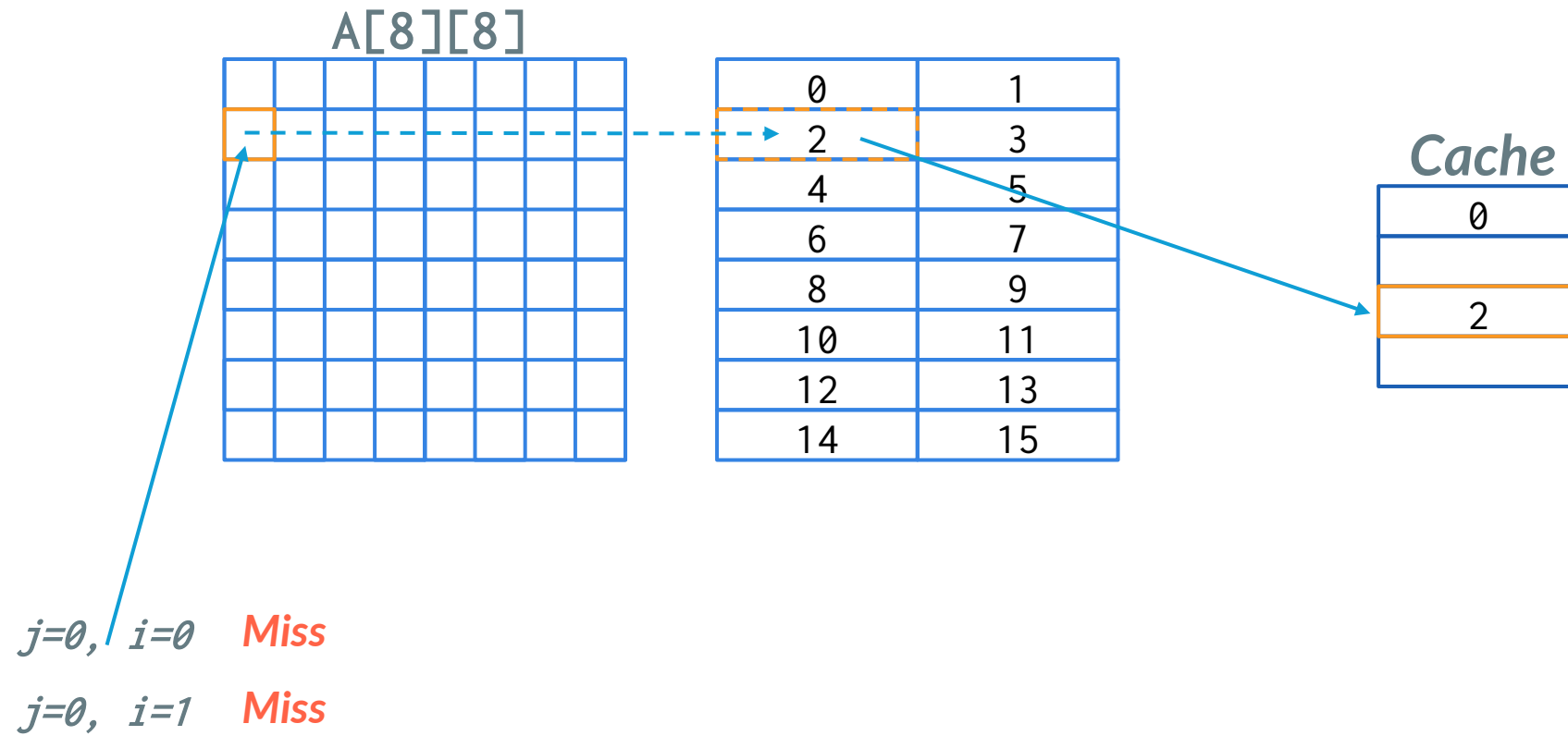


The cache
starts initially
empty

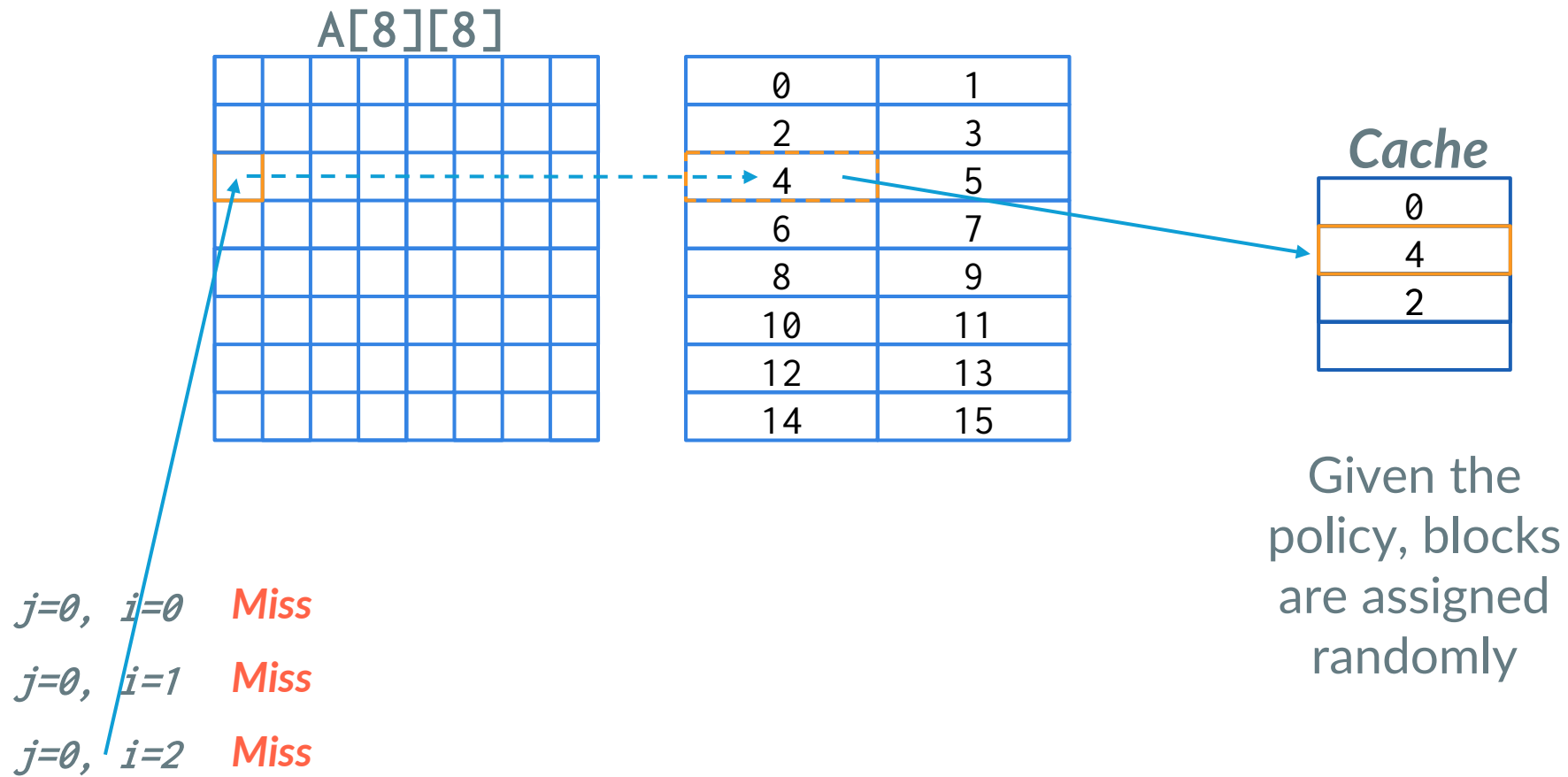
CASE A



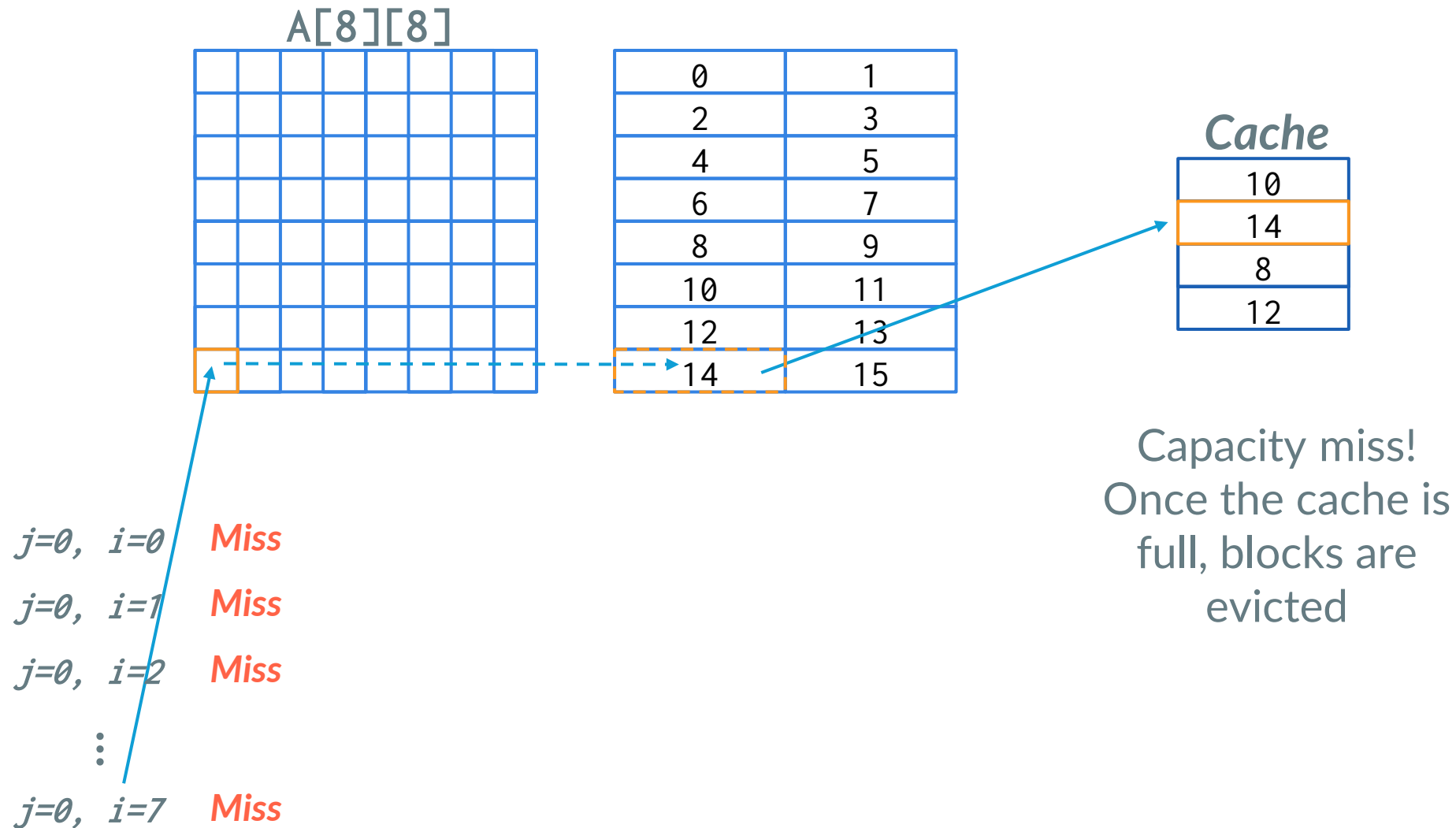
CASE A



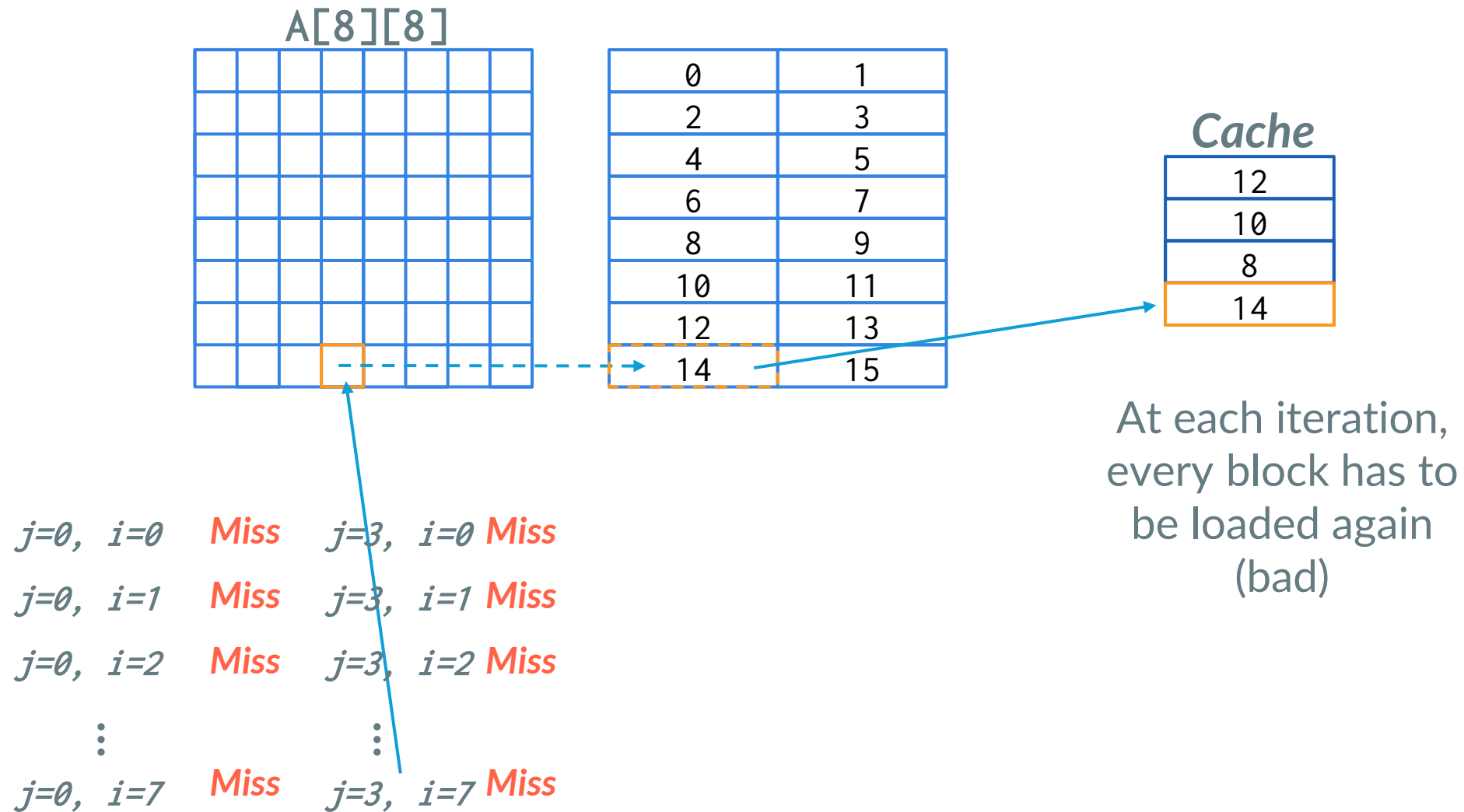
CASE A



CASE A



CASE A



CASE A

A[8][8]

0	1
2	3
4	5
6	7
8	9
10	11
12	13
14	15

Cache

9
13
15
11

$j=0, i=0$	Miss	$j=3, i=0$	Miss	$j=7, i=0$	Miss
$j=0, i=1$	Miss	$j=3, i=1$	Miss	$j=7, i=1$	Miss
$j=0, i=2$	Miss	$j=3, i=2$	Miss	$j=7, i=2$	Miss
$\vdots$		$\vdots$		$\vdots$	
$j=0, i=7$	Miss	$j=3, i=7$	Miss	$j=7, i=7$	Miss

At no point in the process was the cache utilized effectively

EXAMPLE: CACHE MISSES IN DOUBLY NESTED LOOP

Assume a fully-associative cache can hold 4 cache blocks. Each cache block can hold 4 single precision floating point numbers such that the cache can hold 16 floats in total. Determine the number of any *cache misses* and the *hit rate* in the following C++ codes:

Case A:

```
float A[8][8];    // 64 float numbers
float sum = 0.0;

for (int j = 0; j < 8; ++j) {
    for (int i = 0; i < 8; ++i) {
        sum += A[i][j];
    }
}
```

Case B:

```
float A[8][8];    // 64 float numbers
float sum = 0.0;

for (int i = 0; i < 8; ++i) {
    for (int j = 0; j < 8; ++j) {
        sum += A[i][j];
    }
}
```

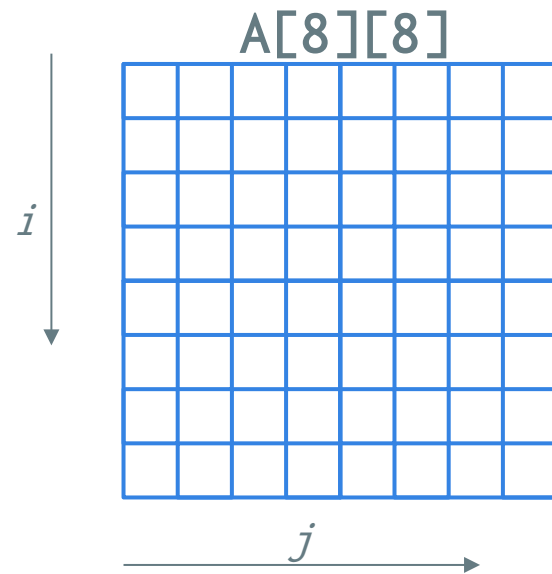
C/C++ uses **row-major** order

Access pattern has *bad* spatial locality

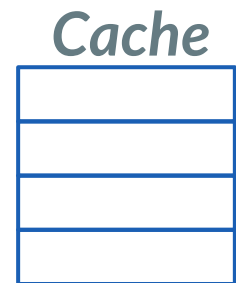
with **64** cache misses

$$\frac{64-64}{64} \times 100\% = 0\% \text{ hit rate.}$$

CASE B

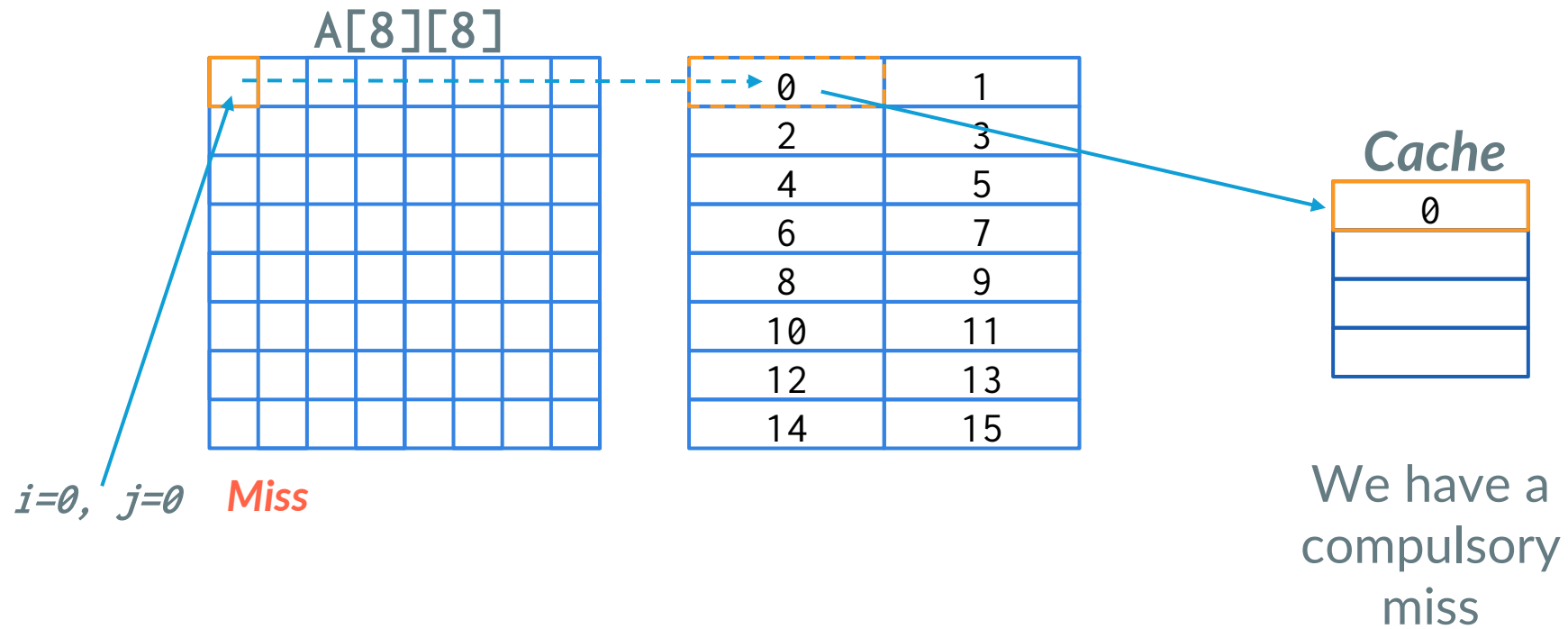


0	1
2	3
4	5
6	7
8	9
10	11
12	13
14	15

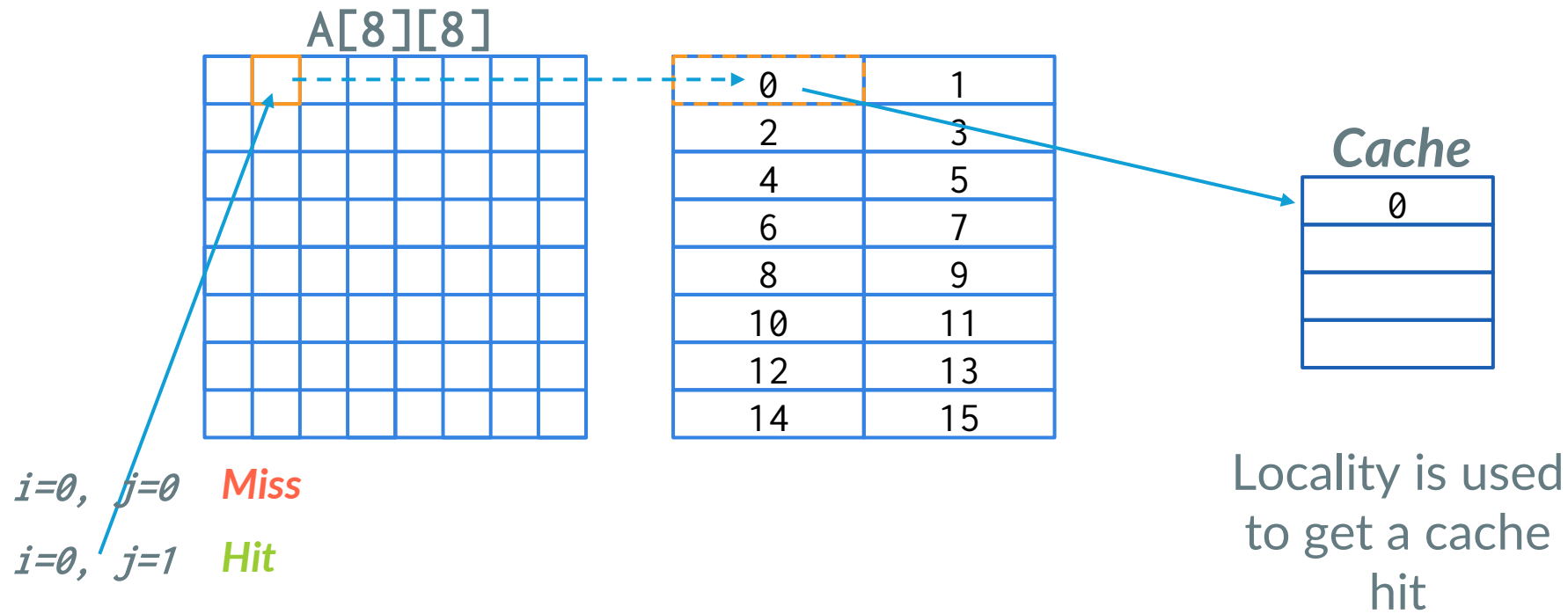


The cache
starts initially
empty

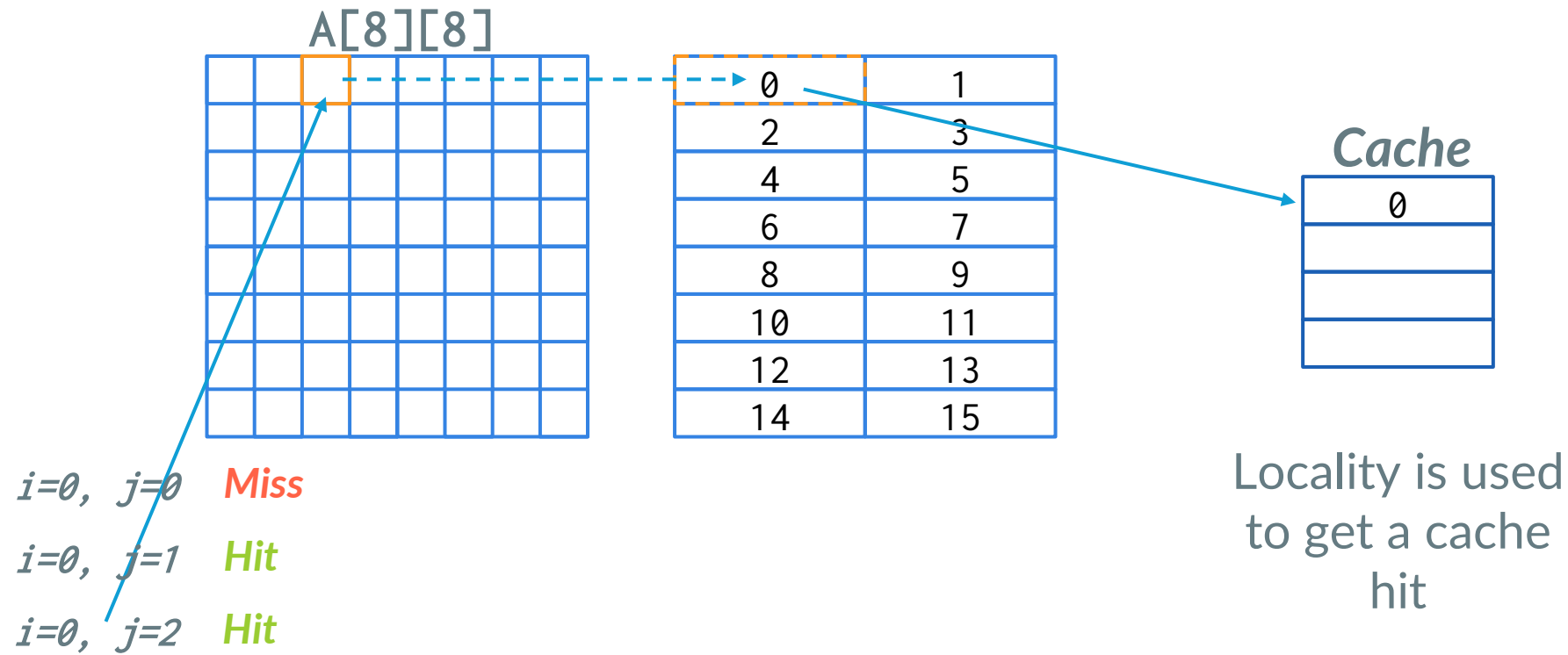
CASE B



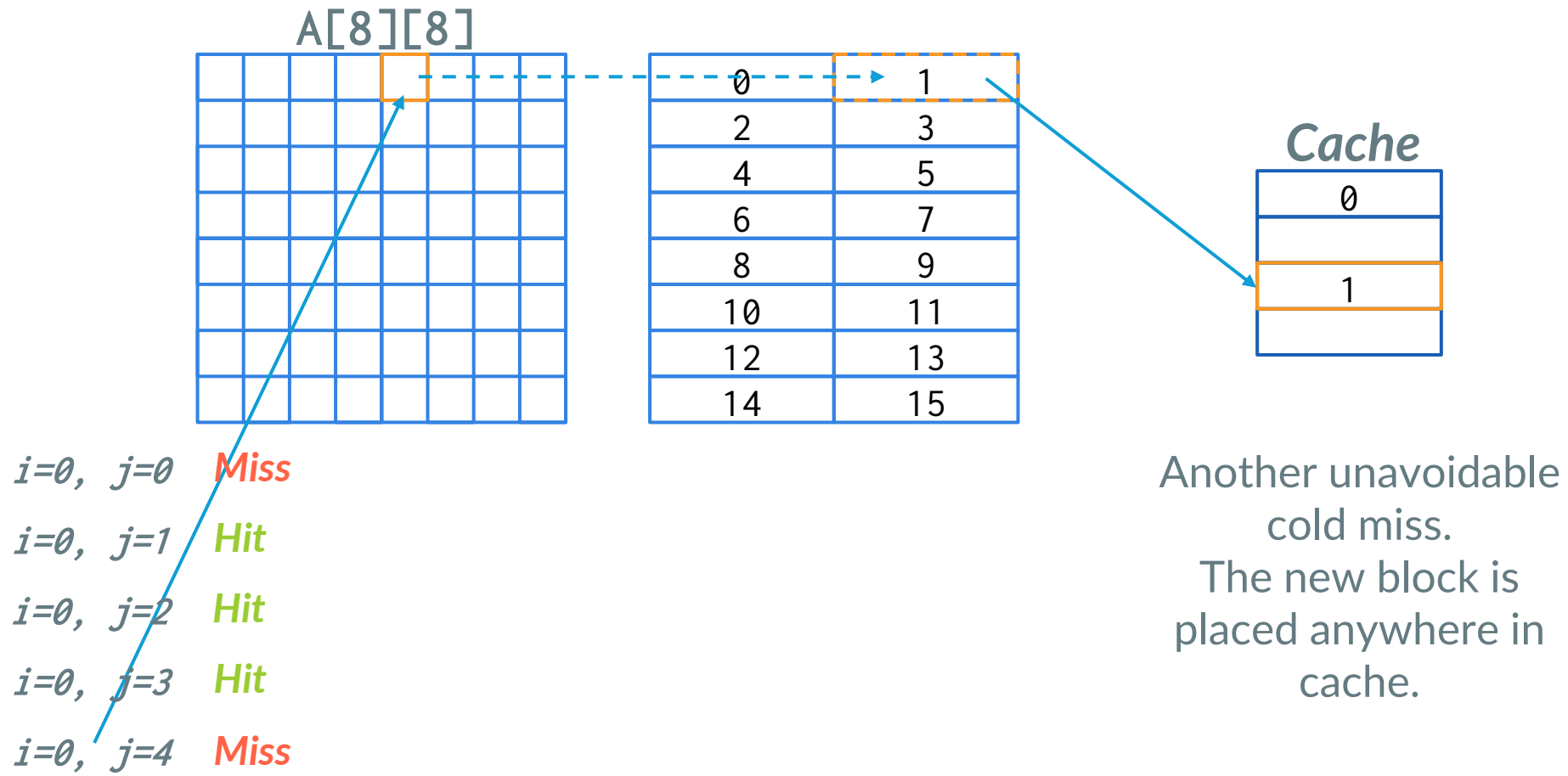
CASE B



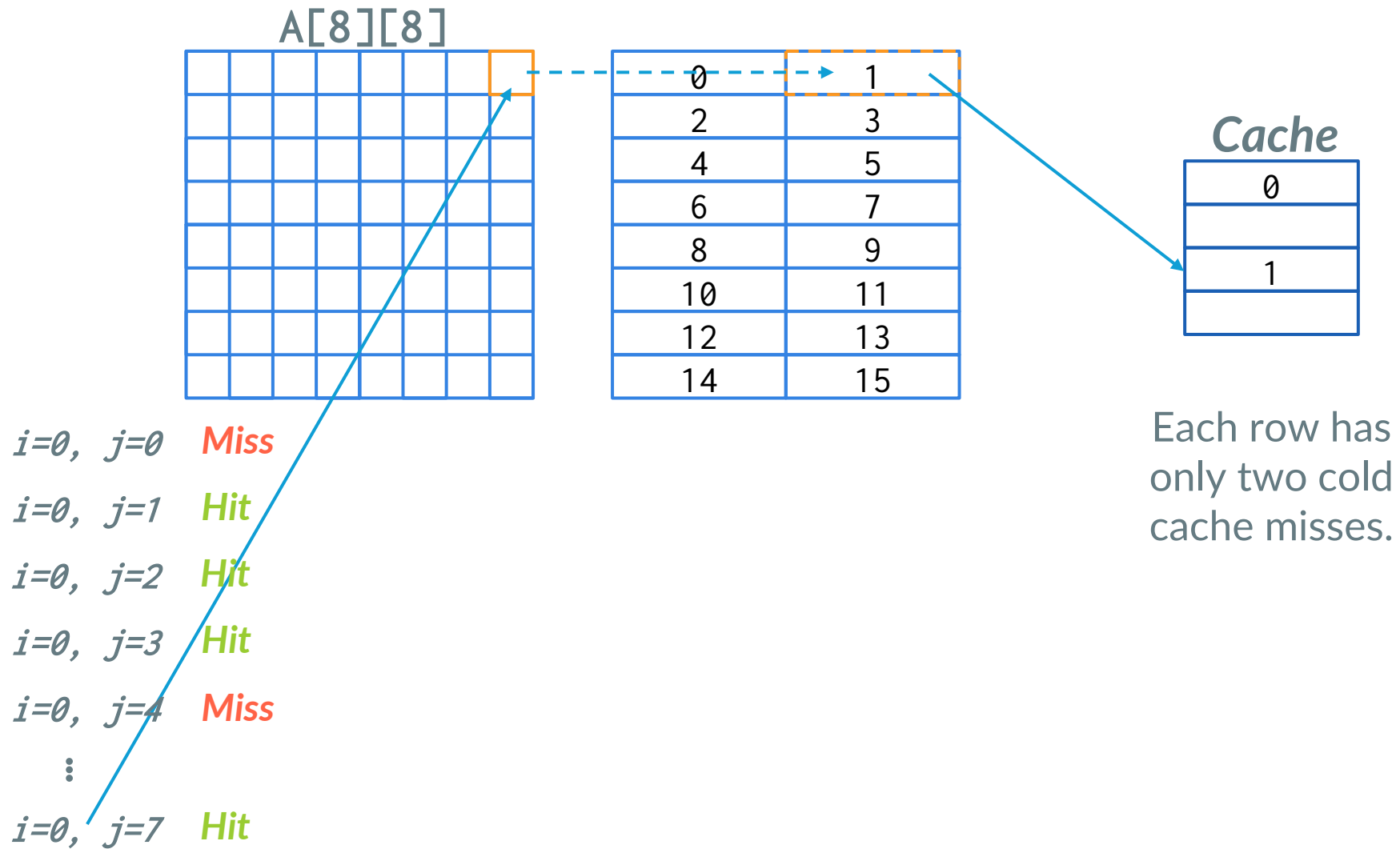
CASE B



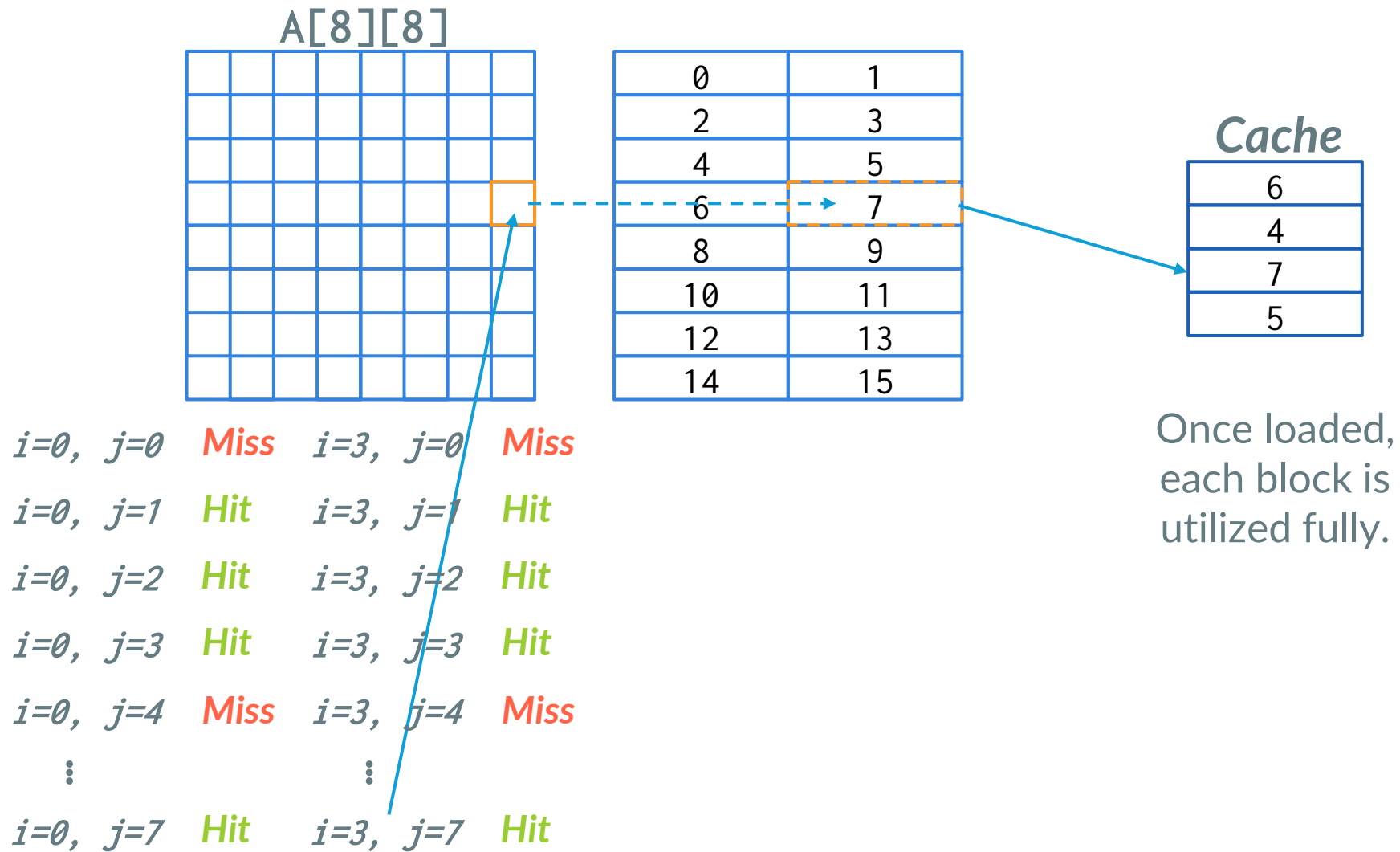
CASE B



CASE B



CASE B



CASE B

A[8][8]

0	1
2	3
4	5
6	7
8	9
10	11
12	13
14	15

Cache

13
15
14
12

$i=0, j=0$	Miss	$i=3, j=0$	Miss	$i=7, j=0$	Miss
$i=0, j=1$	Hit	$i=3, j=1$	Hit	$i=7, j=1$	Hit
$i=0, j=2$	Hit	$i=3, j=2$	Hit	$i=7, j=2$	Hit
$i=0, j=3$	Hit	$i=3, j=3$	Hit	$i=7, j=3$	Hit
$i=0, j=4$	Miss	$i=3, j=4$	Miss	$i=7, j=4$	Miss
$\vdots$		$\vdots$		$\vdots$	
$i=0, j=7$	Hit	$i=3, j=7$	Hit	$i=7, j=7$	Hit

This case utilized the cache optimally.

EXAMPLE: CACHE MISSES IN DOUBLY NESTED LOOP

Assume a fully-associative cache can hold 4 cache blocks. Each cache block can hold 4 single precision floating point numbers such that the cache can hold 16 floats in total. Determine the number of any *cache misses* and the *hit rate* in the following C++ codes:

Case A:

```
float A[8][8];    // 64 float numbers
float sum = 0.0;

for (int j = 0; j < 8; ++j) {
    for (int i = 0; i < 8; ++i) {
        sum += A[i][j];
    }
}
```

C/C++ uses **row-major** order
Access pattern has *bad* spatial locality
with **64** cache misses

$$\frac{64-64}{64} \times 100\% = 0\% \text{ hit rate.}$$

Case B:

```
float A[8][8];    // 64 float numbers
float sum = 0.0;

for (int i = 0; i < 8; ++i) {
    for (int j = 0; j < 8; ++j) {
        sum += A[i][j];
    }
}
```

C/C++ uses **row-major** order
Access pattern has *good* spatial locality
16 cache misses

$$\frac{64-16}{64} \times 100\% = 75\% \text{ hit rate.}$$

IMPACT OF CACHE AND BLOCK SIZES

Impact of cache size

A larger cache will tend to increase the hit rate, but it is harder to make large memories run fast. As a result, larger caches tend to increase the *hit time*. This is the reason why an L1 cache is smaller than an L2 and L2 is smaller than L3.

Impact of block (or line) size

Large blocks are good and bad. Larger blocks help increase the hit rate by exploiting spatial locality. Larger blocks also means that fewer blocks fit into the cache which can hurt the hit rate in programs with high fraction of *temporal locality*. A trade off on modern systems is usually 64 byte per cache block.

RECAP

All computers employ a hierarchy of caches to improve bandwidth by *exploiting locality*

Programmers aware of cache memories improve the performance of their programs

Principle of locality:

- Temporal locality: reuse the same data element nearby in time

- Spatial locality: reference data elements that are nearby in memory

Caches and cache lines/blocks

NEXT TIME: SIMD VECTORIZATION

We say a code is *vectorized*
when it processes vectors of
data with a single instruction

FLOATING POINT INSTRUCTIONS

Single	Double	Operation	Description	Skylake	Broadwell
vaddss	vaddsd	$\text{dst} \leftarrow \text{src2} + \text{src1}$	Floating-point add	4/4	3/3
vsubss	vsubsd	$\text{dst} \leftarrow \text{src2} - \text{src1}$	Floating-point subtract	4/4	3/3
vmulss	vmulsd	$\text{dst} \leftarrow \text{src2} * \text{src1}$	Floating-point multiply	4/4	3/5
vdivss	vdivsd	$\text{dst} \leftarrow \text{src2} / \text{src1}$	Floating-point divide	11/14	<11/<14
vmaxss	vmaxsd	$\text{dst} \leftarrow \max(\text{src2}, \text{src1})$	Floating-point maximum	4/4	3/3
vminss	vminsd	$\text{dst} \leftarrow \min(\text{src2}, \text{src1})$	Floating-point minimum	4/4	3/3
vsqrtss	vsqrtsd	$\text{dst} \leftarrow \sqrt{\text{src1}}$	Floating-point square root	12/18	13/20

- These instructions are SISD. The last two characters in the instruction name stand for *scalar single (ss)* or *scalar double (sd)*.
- *Scalar* means *scalar data* (one element for the corresponding precision).
- SIMD instructions work on *packed data*, where packed refers to multiple data elements in the (packed) vector register.

OVERVIEW OF X86 SIMD EXTENSIONS

	Vector register width	Microarchitecture (Intel)	Time
Multimedia Extensions (MMX)	64-bit (integers)	x86-64, 64-bit	1985
		IA32, 32-bit	1997
Streaming SIMD Extensions (SSE)	128-bit	386, 486, Pentium	1999
		MMX	
		SSE	Pentium III
		SSE2	Pentium 4
Advanced Vector Extensions (AVX)	256-bit	SSE3	Pentium 4E
		SSE4	Pentium 4F
			Core 2 Duo
			Core i7 (Nehalem)
		AVX	2006
	512-bit	AVX2	2008
			2011
		AVX512	2013
			2015
			2019

Run `lscpu` to get a listing of what your processor supports.

FORESHADOWING: WHY GPUS?

WHY GPUS?

- Recall: challenges sustaining Moore's Law
 - Difficult to further increase transistor density and clock rate without material meltdown
- Recall: multicore era since ~2004
 - increase number of transistors by increasing core count
- GPUs are data parallel by design → rendering pixels is massively parallel

CPU core:

- Heavyweight
- *Cache oriented*
- More transistors dedicated to control and logic (good all-rounder core)

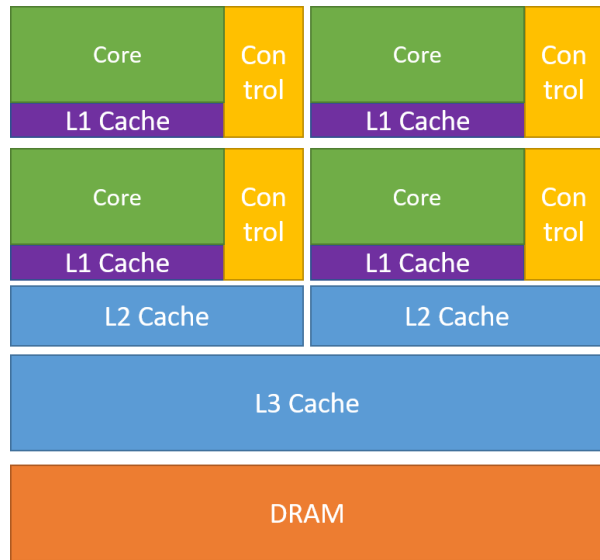
GPU core:

- Lightweight
- Runs at *lower* clock rates
- More transistors dedicated to data processing (less flexible core)
- Requires *much higher memory bandwidth* to service all the cores!

WHY GPUS?

A GPU provides *much higher instruction throughput and memory bandwidth* than a CPU within a similar price and power envelope.

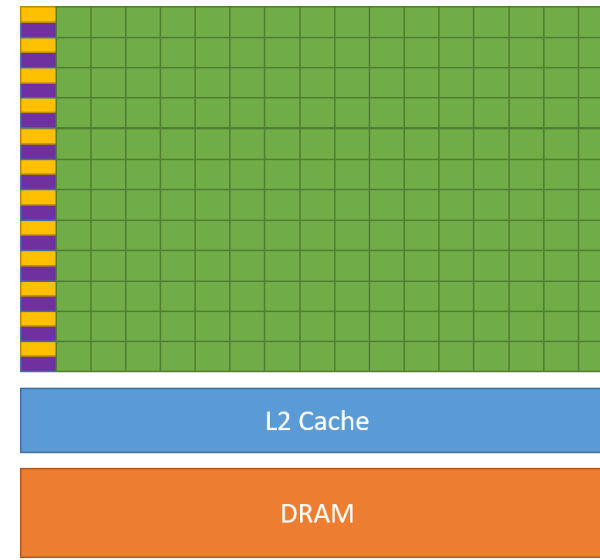
CPU architecture:



CPU

- Optimized for low-latency tasks with heavy usage of fast cache hierarchies.
- More transistors dedicated for control and logic.
- Fat cores.

GPU architecture:



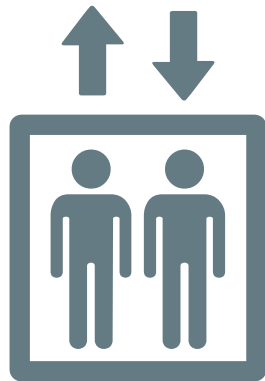
GPU

- Optimized for high instruction throughput.
- More transistors dedicated to data processing (many light cores).

LATENCY AND THROUGHPUT

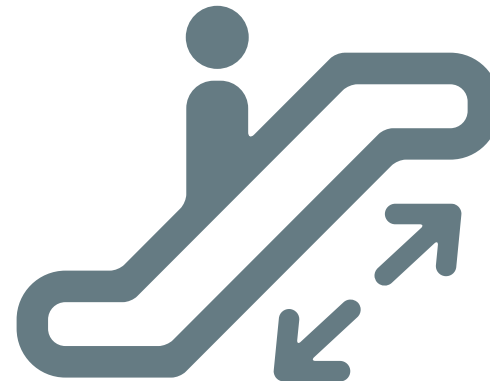
Latency:

- Latency *denotes a waiting time*. Latency is given in units of time.
- Does not depend on the workload.
- Characteristics of *low latency*: low mass (inertia); agile; adaptivity; offense in sports.
- Characteristics of *high latency*: high mass (inertia); sluggish; defense in sports.
- CPU architectures are optimized for latency.



Throughput or Bandwidth:

- Throughput *denotes a rate-based derived metric*
- For the pipe example above, *bandwidth* and *throughput* are interchangeable.
- The throughput tells you how long it takes to process a certain workload.
- GPU architectures are optimized for high throughput.



LATENCY AND THROUGHPUT (MORE EXAMPLES)

Workshop press:



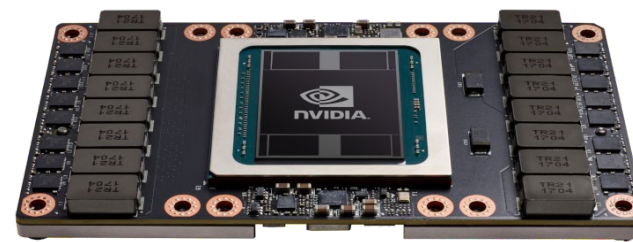
Optimized for latency → can work on many different plate formats and can put them in and out of the press quickly.



Continuous press (plant):



Optimized for throughput → takes time to change plate format. Once setup you get a lot of plates per unit time.



EXAMPLE: EMPTY SHOPPING MALL

- Consider 3 friends in an empty shopping mall who want to go one level up.
- Elevator option:
 - Latency: 8 seconds
 - Capacity: 8 persons
- Escalator option:
 - 2 lanes
 - Latency: 15 seconds
 - Throughput: 2 persons/second

Which is the better option in this scenario: elevator or escalator?

- **Elevator:** 8 seconds (**best option**)
- **Escalator:** $15 + 1 = 16$ seconds



EXAMPLE: BUSY SHOPPING MALL

- Consider 3000 friends in a shopping mall. Everyone wants to go up one level

Elevator time-to-solution:

$$8 \text{ s} + \frac{3000 - 8}{8 \text{ s}} \times 16 \text{ s} = 5992 \text{ s}$$

Escalator time-to-solution:

$$15 \text{ s} + \frac{3000 - 2}{2 \text{ persons/s}} = 1514 \text{ s}$$

- Escalator is 4 times faster! *Why?*
- Nominal throughput:*
 - *Elevator:* 0.5 person/second
 - *Escalator:* 2 person/second



SIMD VS SIMT

SIMD (Single Instruction, Multiple Data)

- One instruction operates on many data elements **at once**
- All lanes do **exactly the same thing** at the same time
- Common on CPUs (vector units like SSE/AVX)
- Mental model: “One instruction, one program counter, many numbers.”

SIMT (Single Instruction, Multiple Threads)

- You write code as if each **thread** is independent
- Each thread has its **own program counter and registers**
- Hardware groups threads together and executes them in lockstep when possible
- Mental model: “Many lightweight threads, but the hardware runs them like SIMD under the hood.”

SIMD exposes vector hardware directly; SIMT hides SIMD behind a thread abstraction